

VRChat Paper Tablet UI Systeem

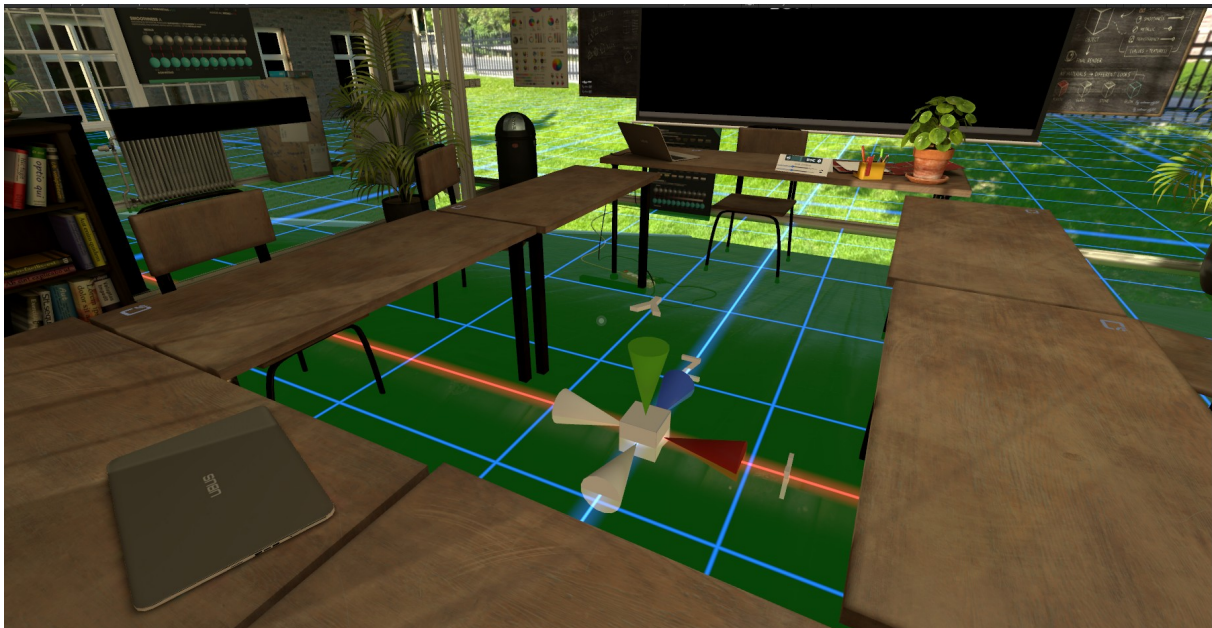
Producthandleiding, architectuurdocument en AI-startprompt

Versie 1.6 - 2026-05-21

Voor StefaniInVR werelden - Open Classroom, cinema, festival, kunstwereld en toekomstige VRChat projecten

Actuele status: het Hybrid Object Toggle systeem v1.5 is getest met de Bird en werkt. Daarna is de Global Material Switch module v1.6 toegevoegd en getest. Deze module kan globaal materialen wisselen, optioneel extra GameObjects aan/uit zetten en de knop-sprite laten meewisselen met dezelfde material index.

Belangrijk: oude screenshots van de scene hierarchy mogen niet als harde bron van waarheid worden gebruikt. Gebruik altijd de huidige scripts, deze handleiding en een nieuwe Scene Analyzer output als exacte hierarchy nodig is.



Screenshot: Global Material Switch test met normale/speciale classroom look en extra grid/gizmo GameObject.

Inhoud

1. Wat dit systeem is
 2. Kernregel en architectuur
 3. Versiegeschiedenis en actuele status
 4. Huidige hierarchy en prefab-denken
 5. Netwerk- en late-join architectuur
 6. Modules en verantwoordelijkheden
 7. Screen Readability module v1.3
 8. Globe module v1.4
 9. Hybrid Object Toggle module v1.5
 10. Global Material Switch module v1.6
 11. Unity setup v1.6 stap voor stap
 12. Testplan en huidige teststatus
 13. Debuggen met de Nova Scene Tablet Analyzer
 14. Veilig uitbreiden naar andere werelden
 15. Veelvoorkomende fouten
 16. AI-startprompt voor toekomstige chats
 17. Script-inventaris
 18. Toekomstige uitbreidingen
- Appendices - volledige scripts

1. Wat dit systeem is

De VRChat Paper Tablet UI is een herbruikbaar tablet-/clipboard-systeem voor VRChat worlds. Het systeem is bedoeld voor world-space UI in de Open Classroom, maar de structuur is bewust product-achtig opgezet zodat modules later ook in andere StefaniInVR werelden kunnen worden gebruikt.

- Bezoekers bedienen publieke functies via duidelijke tabs, knoppen en sliders.
- Hosts, VIPs en admins kunnen extra functies krijgen via lokale access filtering.
- Managers staan altijd actief onder UIs/Managers en bewaren de waarheid.
- Buttons en sliders geven alleen input door.
- Visuals, sprites, renderers en materialen worden lokaal opnieuw opgebouwd op basis van manager-state.
- Late joiners krijgen de juiste globale status via synced managerwaarden.
- Het systeem blijft Quest-vriendelijk door alleen compacte waarden te syncen.

2. Kernregel en architectuur

Hoofdreel: Manager = waarheid. Button = input. Visual = lokale weergave.

Onderdeel	Rol	Mag netwerkwaarheid bewaren?
Manager	Bewaart synced/global state en refresht targets/visuals.	Ja
Input/Button	Roept een functie op de manager aan.	Nee
Target/Object	Past lokale weergave toe, zoals SetActive of material switch.	Nee
Visual/Sprite	Toont de huidige effectieve staat.	Nee

Deze scheiding is belangrijk omdat VRChat multiplayer snel rommelig wordt als UI-objecten zelf state bewaren. Met deze aanpak is debuggen eenvoudiger: als iets niet klopt, kijk eerst naar de manager-state en daarna pas naar targets en visuals.

3. Versiegeschiedenis en actuele status

Versie	Status	Belangrijkste toevoeging
v1.0-v1.2	Historisch	Basis tablet, tabs, toggles, reset, VIP en classroom tools.
v1.3	Actief/historisch	ScreenReadabilityManager vervangt oude brightness/contrast sliderlogica.
v1.4	Actief	GlobeTransformManager: global yaw, global scale, UI handle, resetgedrag.
v1.5	Actief	Hybrid Object Toggle: Bird global/local override met sprite-toggle visuals.
v1.6	Actief / nieuwste	Global Material Switch: synced material index, extra object toggles en knop-sprite switch.

v1.6 is nu de nieuwste handleiding. Oudere versies blijven nuttig om de ontwikkeling te begrijpen, maar de huidige scripts en deze v1.6 handleiding zijn leidend.

4. Huidige hierarchy en prefab-denken

De precieze scene hierarchy kan veranderen. Gebruik oude screenshots alleen als visuele hulp, niet als absolute waarheid. De vaste regel blijft dat managers onder UIs/Managers staan en altijd actief blijven.

- | - UIs
 - | | - Managers
 - | | | - VipAccessManager
 - | | | - TXL Screen Auto Visibility
 - | | | - Projector Screen Toggle
 - | | | - Tab Manager
 - | | | - Toggle Manager
 - | | | - Reset Objects Manager
 - | | | - ScreenReadabilityManager
 - | | | - GlobeTransformManager
 - | | | - HybridObjectToggleManager
 - | | | - GlobalMaterialSwitchManager
 - | | - Paper Tablet System
 - | | - Table System
 - | | - Public Player Controls
 - | | - Other Toggles and Systems
- Managers moeten altijd actief blijven.
- Managers staan op layer Default, niet op Pickup.
- Buttons, sliders en visuals mogen onder panels of tools staan, maar bewaren geen waarheid.
- Pickup roots mogen niet onnodig geschaald worden.
- Voor fysieke objecten: houdt VRCObjectSync, Collider, Rigidbody en visual roots bewust gescheiden als scale of material logic nodig is.

5. Netwerk- en late-join architectuur

Het systeem syncet alleen kleine managerwaarden. Alles wat de speler ziet, wordt lokaal gereconstrueerd nadat synced data binnenkomt.

- Gebruik Manual sync voor managers die globale state bewaren.
- Gebruik RequestSerialization pas nadat ownership veilig is overgenomen.
- Gebruik OnDeserialization om targets en visuals opnieuw te refreshen.
- Gebruik geen Update loops voor simpele toggles/switches.
- Sync geen sprites, materials, renderers of UI-objecten zelf.
- Late join proof betekent: de nieuwe speler ontvangt manager-state en past daarna lokaal objecten/materials/sprites toe.

6. Modules en verantwoordelijkheden

Module	Scripts	Waarvoor
Tabs	PaperTabletTabManager, PaperTabletTabButton, PaperTabletTabState	Gloobaal actieve tabs en lokale visual states.
Toggles	PaperTabletToggleManager, PaperTabletToggleButton, PaperTabletToggleVisual	Bestaande normale global/local toggles.
VIP / lock / blocker	VipAccessManager	Lokale access filtering, tablet lock en stage blocker.
Reset	ResetObjectManager, ResettableObject	Objecten terugzetten naar home transform.
Screen readability	ScreenReadabilityManager, ScreenReadabilityInput	Brightness/contrast global en local met resetgedrag.
Globe	GlobeTransformManager	Global yaw en scale voor globe, plus reset controls.
Hybrid object toggles	HybridObjectToggleManager, HybridObjectToggleInput, HybridObjectToggleGroup	Global aan/uit plus local override per speler.
Global material switch	GlobalMaterialSwitchManager, GlobalMaterialSwitchInput,	Global material index, extra objects en sprite switch.

Module	Scripts	Waarvoor
	GlobalMaterialSwitchTarget	

7. Screen Readability module v1.3

Brightness en contrast zijn sinds v1.3 ondergebracht in ScreenReadabilityManager. Het oude PaperTabletSliderManager systeem mag eventueel blijven bestaan voor legacy/general sliders, maar is niet meer de waarheid voor screen brightness/contrast.

- Global brightness/contrast op hoofdtabelt werkt voor de relevante screens.
- Local brightness/contrast op table screens en local player controller werkt per speler.
- Global reset overrulet local readability waarden.
- Local reset reset alleen de lokale speler.
- Late join is getest en werkt.



Local Player Controller met lokale readability bediening.

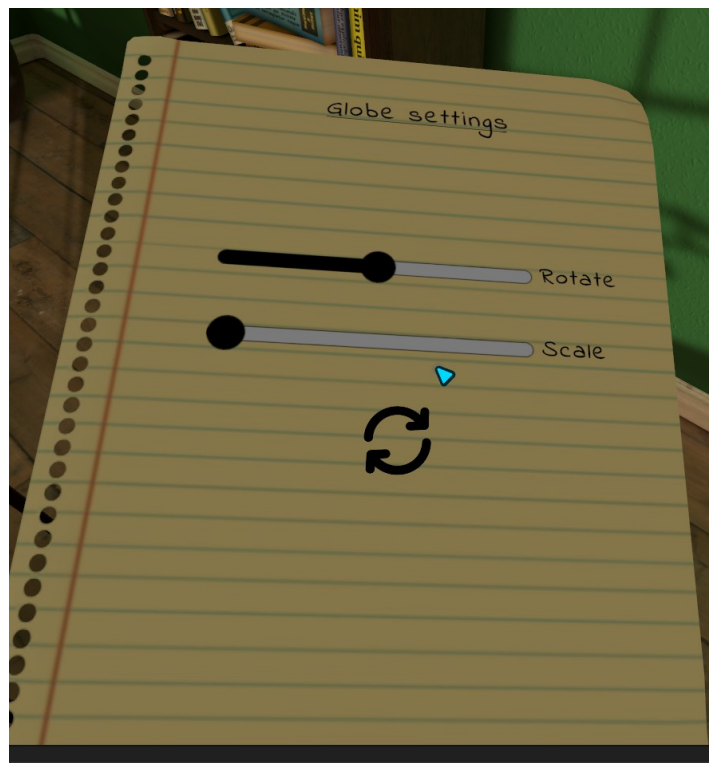
8. Globe module v1.4

De globe module maakt van de wereldbol een bruikbaar classroom-object. De globe kan fysiek worden opgepakt, de UI kan via een losse handle worden verplaatst en yaw/scale worden globaal gesynced.

- GlobeTransformManager staat onder UIs/Managers en blijft actief.
- Globe_toy is de fysieke pickup root met VRC_Pickup, Rigidbody, Collider en VRCObjectSync.
- GlobeScaleRoot is alleen de visual scale root en wordt door de scale slider geschaald.
- Globe_Pivot is de yaw pivot en wordt door de rotate slider gedraaid.
- Default Yaw 01 = 0.5 betekent 0 graden bij range -180 tot 180.
- Default Scale 01 = 0 betekent normale schaal 1 wanneer Min Scale = 1.
- Reset bestaat uit twee delen: ResettableObject voor positie/rotatie en ResetGlobeControls voor yaw/scale sliders.



Globe object in de classroom.



Globe UI met rotate en scale controls.

9. Hybrid Object Toggle module v1.5

Deze module is gebouwd voor objecten die zowel globaal als lokaal bediend moeten kunnen worden. De eerste test was de Bird.

Onderdeel	Script	Rol
Manager	HybridObjectToggleManager	Bewaart global state bits en global revision.
Input	HybridObjectToggleInput	Button die global of local actie doorgeeft.
Group	HybridObjectToggleGroup	Past object active state toe en wisselt

Onderdeel	Script	Rol
		global/local sprites.

- Global button verandert de gesyncte global state.
- Local button verandert alleen de lokale effectieve zichtbaarheid voor deze speler.
- Een nieuwe global actie wist/negeert lokale overrides via globalRevision.
- Global sprite toont de globale waarheid.
- Local sprite toont de lokale effectieve staat.
- Voor de Bird is getest dat local afwijking werkt en dat global weer iedereen overrulet.



Bird object als eerste testobject voor Hybrid Object Toggle.



Local Bird toggle met sprite-based ON/OFF visual.

10. Global Material Switch module v1.6

De Global Material Switch module wisselt wereldwijd dezelfde material stand. Daarnaast kan per material index een extra GameObject aan/uit gezet worden en kan de UI-knop een sprite wisselen die bij dezelfde index hoort.

10.1 Waarom dit apart is van Hybrid Object Toggle

Hybrid Object Toggle is bedoeld voor aan/uit met eventueel lokale speler-afwijking. Global Material Switch is bedoeld voor een globale index: materiaal 0, materiaal 1, materiaal 2, enzovoort. Daarom is dit een aparte manager.

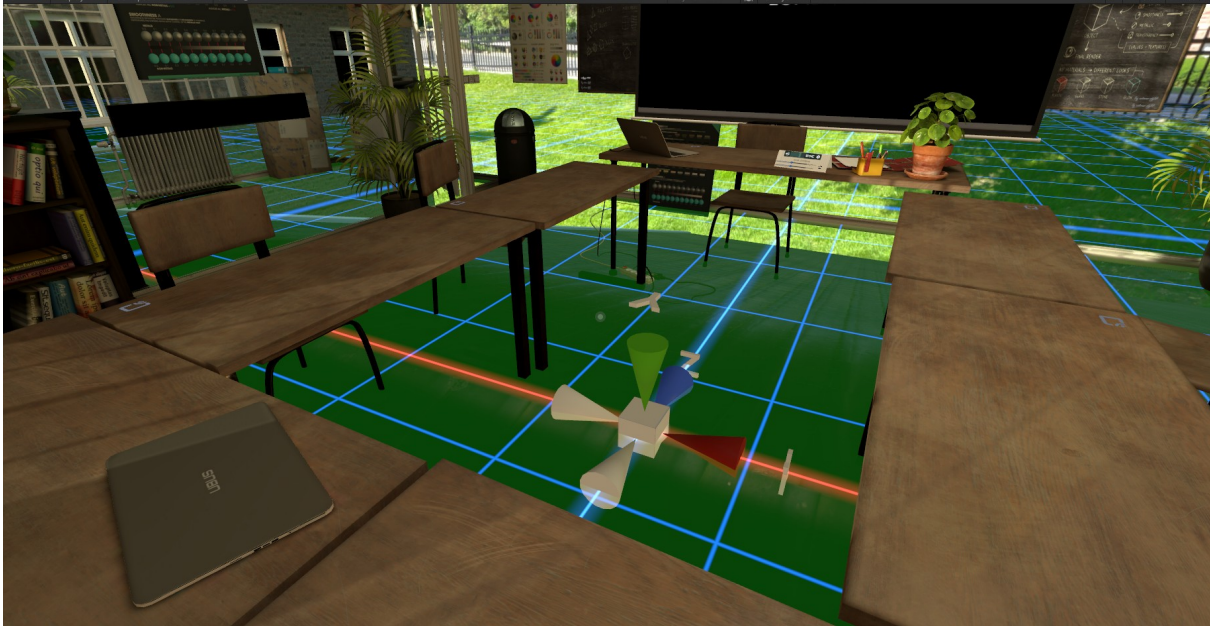
Script	Sync?	Verantwoordelijkheid
GlobalMaterialSwitchManager	Ja, Manual sync	Bewaart packedIndexes met material index per groep.
GlobalMaterialSwitchInput	Nee	Knop roept NextMaterial, PreviousMaterial of SetMaterialIndex aan.
GlobalMaterialSwitchTarget	Nee	Past renderer material, extra objects en UI sprite lokaal toe.

10.2 Wat v1.6 nu ondersteunt

- Een globale material index per switch group.
- Maximaal 7 groups in de huidige packed int opzet.
- Maximaal 16 standen per group, index 0 tot en met 15.
- Next knop: 0 -> 1 -> 0 bij twee materials, of rondlopend bij meer materials.
- Previous knop indien nodig.
- Fixed index knop indien nodig.
- Renderer material wisselen via sharedMaterials en materialSlot.
- Extra GameObjects aanzetten bij een gekozen index, bijvoorbeeld grid/gizmo effect bij index 1.
- Extra GameObjects uitzetten bij een gekozen index, indien nodig.
- UI Image sprite wisselen via spritelmage en sprites array.
- Late join proof doordat de manager de material index syncet en targets op OnDeserialization refreshen.

10.3 Voorbeeld: normale classroom look en speciale grid/gizmo look

Index	Material	Extra object	Button sprite
0	Normaal material	Grid/gizmo object uit	Normale sprite
1	Speciaal material	Grid/gizmo object aan	Speciale sprite



v1.6 test: material switch met extra grid/gizmo object en bijbehorende button sprite.

11. Unity setup v1.6 stap voor stap

Deze setup beschrijft de eenvoudige versie: 1 switch group, 2 materials, 1 extra object en 1 knop-sprite die meewisselt.

11.1 Manager

UIs

|- Managers

|- GlobalMaterialSwitchManager

- GlobalMaterialSwitchManager.cs

1. Maak onder UIs/Managers een GameObject: GlobalMaterialSwitchManager.
2. Zet GlobalMaterialSwitchManager.cs erop.
3. Zet Default Indexes size op 1.
4. Zet Element 0 op 0. Dit betekent: start met material index 0.
5. Laat Targets eerst leeg tot het target-object is gemaakt.

11.2 Target

MyObject_MaterialSwitchTarget

- GlobalMaterialSwitchTarget.cs

6. Maak een leeg target-object, bijvoorbeeld Classroom_Grid_MaterialSwitchTarget.
7. Zet GlobalMaterialSwitchTarget.cs erop.
8. Vul Manager = GlobalMaterialSwitchManager.
9. Vul Group Index = 0.
10. Sleep de mesh renderer die van materiaal moet wisselen naar Target Renderer.
11. Laat Material Slot op 0, tenzij het tweede materiaal-slot vervangen moet worden.
12. Zet Materials size op 2: Element 0 = normaal materiaal, Element 1 = speciaal materiaal.
13. Als er een extra object aan moet bij index 1, vul Extra Objects ON At Selected Index met dat object en zet Extra Objects ON Index op 1.
14. Als de knop-sprite moet wisselen, vul Sprite Image met de Image component van de knop en zet Sprites size op 2: Element 0 = normale sprite, Element 1 = speciale sprite.

11.3 Manager target koppelen

15. Ga terug naar GlobalMaterialSwitchManager.

Open Classroom / Paper Tablet UI - generated documentation v1.6

16. Zet Targets size op 1.
17. Sleep het target-object met GlobalMaterialSwitchTarget in Element 0.

11.4 Button

18. Klik op de knop die de material switch moet bedienen.
19. Zet GlobalMaterialSwitchInput.cs op de knop.
20. Vul Manager = GlobalMaterialSwitchManager.
21. Vul Group Index = 0.
22. Vul Material Count = 2.
23. Laat Next Material aan staan.
24. Laat Previous Material en Set Fixed Index uit, tenzij bewust nodig.
25. Koppel de Unity Button OnClick naar GlobalMaterialSwitchInput.Press().

11.5 Belangrijke index-regel

De volgorde van arrays moet gelijk blijven. Index 0 hoort bij index 0. Index 1 hoort bij index 1.

Materials Element 0 = normaal material
Sprites Element 0 = normale sprite
Extra object state = uit als Extra Objects ON Index 1 is

Materials Element 1 = speciaal material
Sprites Element 1 = speciale sprite
Extra object state = aan als Extra Objects ON Index 1 is

12. Testplan en huidige teststatus

12.1 Geteste status v1.6

- Global Material Switch is getest in de Open Classroom setup.
- Material index 0 toont normale material en normale sprite.
- Material index 1 toont speciale material en speciale sprite.
- Extra grid/gizmo GameObject schakelt mee met index 1.
- Terugschakelen naar index 0 zet het extra object weer uit.
- De basis werkt in editor/VR test volgens de gebruiker.
- Bestaande Hybrid Object Toggle, PaperTabletToggleManager en Globe module zijn niet opengebroken voor deze module.

12.2 Multiplayer en late join checklist

- Start wereld: material index moet default 0 zijn.
- Klik material switch: alle spelers moeten dezelfde material stand zien.
- Extra GameObject moet voor alle spelers dezelfde active state krijgen.
- Button sprite moet lokaal bij elke speler de juiste stand tonen na refresh.
- Late joiner moet meteen de huidige material index zien.
- Andere speler moet de knop kunnen bedienen; ownership moet veilig worden overgenomen door de manager.
- Na scene reload moet default index terugkomen volgens Default Indexes.

13. Debuggen met de Nova Scene Tablet Analyzer

De Nova Scene Tablet Analyzer blijft belangrijk voor toekomstige debugging. Als exacte hierarchy, references of missing fields onduidelijk zijn, draai de analyzer en upload de output samen met de huidige scripts.

- Controleer of managers onder UIs/Managers staan en actief blijven.
- Controleer of manager arrays gevuld zijn: Groups, Targets, enzovoort.

- Controleer of buttons naar Press() wijzen en niet naar oude functies.
- Controleer of target renderers, materials, sprites en extra objects goed gekoppeld zijn.
- Controleer of oude hierarchy screenshots niet worden gebruikt als harde bron van waarheid.

14. Veilig uitbreiden naar andere werelden

- Nieuwe global status hoort op een altijd actieve manager onder UIs/Managers.
- Gebruik aparte modules voor aparte verantwoordelijkheden.
- Breek bestaande werkende managers niet open als een nieuwe losse manager veiliger is.
- Houd input scripts dom: alleen Press() of een kleine doorgeef functie.
- Houd target scripts lokaal: renderer/material/sprite/object state toepassen.
- Test elke nieuwe module eerst met 1 object, daarna met meerdere objecten, daarna pas documenteren.
- Maak altijd een backup na een bewezen werkende staat.

15. Veelvoorkomende fouten

Probleem	Waarschijnlijke oorzaak	Oplossing
Knop doet niets	OnClick niet gekoppeld aan Press() of manager ontbreekt.	Koppel Button OnClick naar Input.Press() en vul manager.
Material wisselt niet	Target Renderer of Materials array ontbreekt.	Vul renderer, material slot en materials array.
Verkeerd material slot verandert	Material Slot staat verkeerd.	Gebruik 0 voor eerste slot, 1 voor tweede slot.
Sprite wisselt niet	Sprite Image of Sprites array ontbreekt.	Vul Image component en sprites in dezelfde volgorde als materials.
Extra object blijft aan	Default active state wordt pas bij Refresh aangepast of indexveld staat verkeerd.	Check Extra Objects ON Index en target in manager Targets array.
Late join klopt niet	Manager synced niet of target refresht niet na deserialization.	Controleer Manual sync, RequestSerialization en OnDeserialization.
Local override blijft hangen bij Bird	Global action of globalRevision niet geraakt.	Gebruik global button via HybridObjectToggleInput en manager.

16. AI-startprompt voor toekomstige chats

Ik werk aan mijn VRChat Paper Tablet UI systeem voor mijn Open Classroom world.

Gebruik de nieuwste handleiding VRChat_Paper_Tablet_UI_Product_Handleiding_2026-05-21_v1_6 als bron van waarheid. Oudere versies v1.0 tot en met v1.5 zijn historisch om de ontwikkeling te begrijpen.

Projectcontext:

- Unity 2022.3.22f1
- VRChat SDK
- UdonSharp
- Quest-vriendelijk
- world-space UI
- Managers staan onder UIs/Managers
- Managers moeten altijd actief blijven
- Managers staan op layer Default, niet Pickup
- Buttons, sliders en visuals bewaren geen netwerkwaarheid
- Hoofdregel: manager = waarheid, button/input = doorgeven, visual/object = lokale weergave
- Geen gokjes: als iets onzeker is, zeg dat eerlijk
- Verander bestaande werkende scripts zo weinig mogelijk
- Geef bij code altijd het volledige script terug
- Werk stap voor stap en leg eerst kort uit waarom

Werkende status t/m v1.6:

- Tabs werken
- Toggles werken
- VIP access werkt
- Tablet lock werkt
- Stage blocker werkt
- Projector screen toggle werkt
- Reset object systeem werkt
- Visitor guide MP4/playlists werken
- ScreenReadabilityManager werkt
- Global brightness/contrast op hoofdtablet werkt
- Local brightness/contrast op table screens werkt
- Local brightness/contrast op local player controller werkt
- Global reset overruled local readability waarden
- Local reset reset alleen lokale speler
- Late join is getest en werkt
- Globe systeem v1.4 is voltooid en getest in VR
- Hybrid Object Toggle systeem v1.5 is getest met de Bird
- Global Material Switch systeem v1.6 is getest met materiaal normaal/speciaal, extra GameObject en sprite switch

Belangrijke modules:

- ScreenReadabilityManager.cs en ScreenReadabilityInput.cs zijn de waarheid voor brightness/contrast
- GlobeTransformManager.cs is de waarheid voor globe yaw en scale
- HybridObjectToggleManager/Input/Group regelen global/local object toggles met local override
- GlobalMaterialSwitchManager/Input/Target regelen global material index, extra object toggles en sprite switch

Belangrijke waarschuwing:

Oude screenshots van de scene hierarchy zijn niet altijd actueel. Gebruik de huidige scripts, deze handleiding en zo nodig een nieuwe Scene Analyzer output.

17. Script-inventaris

Script	Rol
GlobalMaterialSwitchInput.cs	v1.6 Global Material Switch module.
GlobalMaterialSwitchManager.cs	v1.6 Global Material Switch module.
GlobalMaterialSwitchTarget.cs	v1.6 Global Material Switch module.
GlobeTransformManager.cs	v1.4 Globe yaw/scale manager.
HybridObjectToggleGroup.cs	v1.5 Hybrid Object Toggle module.
HybridObjectToggleInput.cs	v1.5 Hybrid Object Toggle module.
HybridObjectToggleManager.cs	v1.5 Hybrid Object Toggle module.
PaperTabletTabButton.cs	Core tab module.
PaperTabletTabManager.cs	Core tab module.
PaperTabletTabState.cs	Core tab module.
PaperTabletToggleButton.cs	Core toggle module.
PaperTabletToggleManager.cs	Core toggle module.
PaperTabletToggleVisual.cs	Core toggle module.
ProjectorScreenToggle.cs	Projector screen controls.
ResetObjectManager.cs	Reset module.
ResettableObject.cs	Reset module.
ScreenReadabilityInput.cs	v1.3 screen readability module.
ScreenReadabilityManager.cs	v1.3 screen readability module.
TXLScreenAutoVisibility.cs	VideoTXL screen visibility helper.
VipAccessManager.cs	VIP, tablet lock and access filtering.

18. Toekomstige uitbreidingen

- Multiple material switch groups voor meer classroom looks of tool-modes.
- Een kleine label/tekst visual voor material index naam, zoals Normal / Special / Grid.
- Een centrale documentatie-update naar v1.7 zodra nieuwe modules getest zijn.
- Prefab-cleanup voor verkoopbare of herbruikbare Open Classroom UI module.
- Extra Scene Analyzer checks voor GlobalMaterialSwitchManager targets en missing sprite/material references.

Appendices - volledige scripts

Deze appendices bevatten de huidige scripts uit Scripts(13).zip. Bij toekomstige codewijzigingen altijd het volledige script teruggeven en deze handleiding daarna bijwerken als de test geslaagd is.

Appendix 1 - GlobalMaterialSwitchInput.cs

```

using UdonSharp;
using UnityEngine;

namespace StefanieInVR
{
    [UdonBehaviourSyncMode(BehaviourSyncMode.None)]
    public class GlobalMaterialSwitchInput : UdonSharpBehaviour
    {
        [Header("Required")]
        public GlobalMaterialSwitchManager manager;

        [Tooltip("Which material switch group this button controls. First switch = 0.")]
        public int groupIndex = 0;

        [Tooltip("How many material options this group has. Example: 2 for normal/special.")]
        public int materialCount = 2;

        [Header("Button Action")]
        [Tooltip("If true, this button goes to the next material index.")]
        public bool nextMaterial = true;

        [Tooltip("If true, this button goes to the previous material index.")]
        public bool previousMaterial = false;

        [Tooltip("If true, this button sets a fixed material index.")]
        public bool setFixedIndex = false;

        [Tooltip("Only used when Set Fixed Index is true.")]
        public int fixedMaterialIndex = 0;

        [Header("Debug")]
        public bool debugLogs = false;

        public override void Interact()
        {
            Press();
        }

        public void Press()
        {
            if (manager == null)
            {
                Debug.LogWarning("[GlobalMaterialSwitchInput] No manager assigned.");
                return;
            }

            if (materialCount <= 0)
            {
                Debug.LogWarning("[GlobalMaterialSwitchInput] Material count must be higher than zero.");
                return;
            }

            if (setFixedIndex)
            {
                manager.SetMaterialIndex(groupIndex, fixedMaterialIndex);
            }
            else if (previousMaterial)
            {
                manager.PreviousMaterial(groupIndex, materialCount);
            }
            else
            {
                manager.NextMaterial(groupIndex, materialCount);
            }

            if (debugLogs)

```


VRChat Paper Tablet UI Product Handleiding - v1.6 - StefanieInVR

```
{
  Debug.Log("[GlobalMaterialSwitchInput] Button pressed. Group: " + groupIndex);
}
}
```

Appendix 2 - GlobalMaterialSwitchManager.cs

```

using UdonSharp;
using UnityEngine;
using VRC.SDKBase;

namespace StefanieInVR
{
    [UdonBehaviourSyncMode(BehaviourSyncMode.Manual)]
    public class GlobalMaterialSwitchManager : UdonSharpBehaviour
    {
        [Header("Default Indexes")]
        [Tooltip("One default material index per switch group. Group 0 = first material switch.")]
        public int[] defaultIndexes;

        [Header("Targets")]
        [Tooltip("All material switch targets that should refresh when a global material index changes.")]
        public GlobalMaterialSwitchTarget[] targets;

        [Header("Debug")]
        public bool debugLogs = false;

        [UdonSynced]
        private int packedIndexes = 0;

        [UdonSynced]
        private bool hasInitializedNetworkState = false;

        private const int BitsPerGroup = 4;
        private const int MaxIndexValue = 15;
        private const int MaxGroups = 7;

        private void Start()
        {
            if (!hasInitializedNetworkState)
            {
                packedIndexes = BuildDefaultPackedIndexes();

                if (Networking.LocalPlayer != null && Networking.IsOwner(gameObject))
                {
                    hasInitializedNetworkState = true;
                    RequestSerialization();

                    if (debugLogs)
                    {
                        Debug.Log("[GlobalMaterialSwitchManager] Initialized default indexes.");
                    }
                }
            }

            SendCustomEventDelayedFrames(nameof(RefreshAll), 1);
        }

        public override void OnDeserialization()
        {
            RefreshAll();

            if (debugLogs)
            {
                Debug.Log("[GlobalMaterialSwitchManager] Deserialized and refreshed.");
            }
        }

        public void NextMaterial(int groupIndex, int materialCount)
        {
            if (!IsValidGroupIndex(groupIndex))
            {
                Debug.LogWarning("[GlobalMaterialSwitchManager] NextMaterial failed. Invalid group index: " + groupIndex);
            }
        }
    }
}

```

```

        return;
    }

    if (materialCount <= 0)
    {
        Debug.LogWarning("[GlobalMaterialSwitchManager] NextMaterial failed. Material count is zero.");
        return;
    }

    if (materialCount > MaxIndexValue + 1)
    {
        materialCount = MaxIndexValue + 1;
    }

    int currentIndex = GetMaterialIndex(groupIndex);
    int nextIndex = currentIndex + 1;

    if (nextIndex >= materialCount)
    {
        nextIndex = 0;
    }

    SetMaterialIndex(groupIndex, nextIndex);
}

public void PreviousMaterial(int groupIndex, int materialCount)
{
    if (!IsValidGroupIndex(groupIndex))
    {
        Debug.LogWarning("[GlobalMaterialSwitchManager] PreviousMaterial failed. Invalid group index: " + groupIndex);
        return;
    }

    if (materialCount <= 0)
    {
        Debug.LogWarning("[GlobalMaterialSwitchManager] PreviousMaterial failed. Material count is zero.");
        return;
    }

    if (materialCount > MaxIndexValue + 1)
    {
        materialCount = MaxIndexValue + 1;
    }

    int currentIndex = GetMaterialIndex(groupIndex);
    int previousIndex = currentIndex - 1;

    if (previousIndex < 0)
    {
        previousIndex = materialCount - 1;
    }

    SetMaterialIndex(groupIndex, previousIndex);
}

public void SetMaterialIndex(int groupIndex, int materialIndex)
{
    if (!IsValidGroupIndex(groupIndex))
    {
        Debug.LogWarning("[GlobalMaterialSwitchManager] SetMaterialIndex failed. Invalid group index: " + groupIndex);
        return;
    }

    if (materialIndex < 0)
    {
        materialIndex = 0;
    }
}

```

VRChat Paper Tablet UI Product Handleiding - v1.6 - StefanieInVR

```
if (materialIndex > MaxIndexValue)
{
    materialIndex = MaxIndexValue;
}

TakeOwnershipIfNeeded();

packedIndexes = SetPackedIndex(packedIndexes, groupIndex, materialIndex);

hasInitializedNetworkState = true;

RequestSerialization();
RefreshAll();

if (debugLogs)
{
    Debug.Log("[GlobalMaterialSwitchManager] Set material group " + groupIndex + " to index " + materialIndex);
}
}

public int GetMaterialIndex(int groupIndex)
{
    if (!IsValidGroupIndex(groupIndex))
    {
        return 0;
    }

    return GetPackedIndex(packedIndexes, groupIndex);
}

public void RefreshAll()
{
    if (targets == null)
    {
        return;
    }

    for (int i = 0; i < targets.Length; i++)
    {
        if (targets[i] != null)
        {
            targets[i].RefreshFromManager();
        }
    }
}

private void TakeOwnershipIfNeeded()
{
    if (Networking.LocalPlayer == null)
    {
        return;
    }

    if (!Networking.IsOwner(gameObject))
    {
        Networking.SetOwner(Networking.LocalPlayer, gameObject);
    }
}

private int BuildDefaultPackedIndexes()
{
    int result = 0;

    if (defaultIndexes == null)
    {
        return result;
    }
}
```

VRChat Paper Tablet UI Product Handleiding - v1.6 - StefanieInVR

```
int max = defaultIndexes.Length;

if (max > MaxGroups)
{
    max = MaxGroups;
}

for (int i = 0; i < max; i++)
{
    int value = defaultIndexes[i];

    if (value < 0)
    {
        value = 0;
    }

    if (value > MaxIndexValue)
    {
        value = MaxIndexValue;
    }

    result = SetPackedIndex(result, i, value);
}

return result;
}

private bool IsValidGroupIndex(int groupIndex)
{
    if (groupIndex < 0)
    {
        return false;
    }

    if (groupIndex >= MaxGroups)
    {
        return false;
    }

    return true;
}

private int GetPackedIndex(int packedValue, int groupIndex)
{
    int shift = groupIndex * BitsPerGroup;
    int mask = MaxIndexValue << shift;
    return (packedValue & mask) >> shift;
}

private int SetPackedIndex(int packedValue, int groupIndex, int materialIndex)
{
    int shift = groupIndex * BitsPerGroup;
    int clearMask = ~(MaxIndexValue << shift);
    int cleanedValue = packedValue & clearMask;
    int newBits = materialIndex << shift;

    return cleanedValue | newBits;
}
}
```

Appendix 3 - GlobalMaterialSwitchTarget.cs

```

using UdonSharp;
using UnityEngine;
using UnityEngine.UI;

namespace StefanieInVR
{
    [UdonBehaviourSyncMode(BehaviourSyncMode.None)]
    public class GlobalMaterialSwitchTarget : UdonSharpBehaviour
    {
        [Header("Required")]
        public GlobalMaterialSwitchManager manager;

        [Tooltip("Which material switch group this target listens to. First switch = 0.")]
        public int groupId = 0;

        [Header("Renderer Material Switch")]
        [Tooltip("Renderer that should receive the selected material.")]
        public Renderer targetRenderer;

        [Tooltip("Material slot on the renderer. Usually 0.")]
        public int materialSlot = 0;

        [Tooltip("Materials to choose from. Index 0 = default, Index 1 = second material, etc.")]
        public Material[] materials;

        [Header("Button / UI Sprite Switch")]
        [Tooltip("Optional UI Image that should change sprite when the material index changes.")]
        public Image spriteImage;

        [Tooltip("Sprites to choose from. Index 0 belongs to material index 0, index 1 belongs to material index 1, etc.")]
        public Sprite[] sprites;

        [Header("Extra Objects ON At Selected Index")]
        [Tooltip("These objects turn ON only when the current material index equals Extra Objects ON Index.")]
        public GameObject[] extraObjectsOnAtIndex;

        [Tooltip("Material index where the extra objects should be ON. Example: 1.")]
        public int extraObjectsOnIndex = 1;

        [Header("Extra Objects OFF At Selected Index")]
        [Tooltip("These objects turn OFF only when the current material index equals Extra Objects OFF Index.")]
        public GameObject[] extraObjectsOffAtIndex;

        [Tooltip("Material index where the extra objects should be OFF. Example: 1.")]
        public int extraObjectsOffIndex = 1;

        [Header("Options")]
        public bool refreshOnStart = true;

        [Header("Debug")]
        public bool debugLogs = false;

        private void Start()
        {
            if (refreshOnStart)
            {
                SendCustomEventDelayedFrames(nameof(RefreshFromManager), 1);
            }
        }

        public void RefreshFromManager()
        {
            if (manager == null)
            {
                Debug.LogWarning("[GlobalMaterialSwitchTarget] No manager assigned.");
                return;
            }
        }
    }
}

```



```

    }

    int index = manager.GetMaterialIndex(groupIndex);

    ApplyMaterial(index);
    ApplySprite(index);
    ApplyExtraObjects(index);

    if (debugLogs)
    {
        Debug.Log("[GlobalMaterialSwitchTarget] Refreshed. Group: " + groupIndex + " Index: " + index);
    }
}

private void ApplyMaterial(int index)
{
    if (targetRenderer == null)
    {
        return;
    }

    if (materials == null)
    {
        return;
    }

    if (materials.Length == 0)
    {
        return;
    }

    if (index < 0)
    {
        index = 0;
    }

    if (index >= materials.Length)
    {
        index = materials.Length - 1;
    }

    Material selectedMaterial = materials[index];

    if (selectedMaterial == null)
    {
        return;
    }

    Material[] rendererMaterials = targetRenderer.sharedMaterials;

    if (rendererMaterials == null)
    {
        return;
    }

    if (materialSlot < 0)
    {
        materialSlot = 0;
    }

    if (materialSlot >= rendererMaterials.Length)
    {
        Debug.LogWarning("[GlobalMaterialSwitchTarget] Material slot is outside renderer material array.");
        return;
    }

    if (rendererMaterials[materialSlot] == selectedMaterial)
    {

```

```

        return;
    }

    rendererMaterials[materialSlot] = selectedMaterial;
    targetRenderer.sharedMaterials = rendererMaterials;
}

private void ApplySprite(int index)
{
    if (spriteImage == null)
    {
        return;
    }

    if (sprites == null)
    {
        return;
    }

    if (sprites.Length == 0)
    {
        return;
    }

    if (index < 0)
    {
        index = 0;
    }

    if (index >= sprites.Length)
    {
        index = sprites.Length - 1;
    }

    Sprite selectedSprite = sprites[index];

    if (selectedSprite == null)
    {
        return;
    }

    if (spriteImage.sprite != selectedSprite)
    {
        spriteImage.sprite = selectedSprite;
    }
}

private void ApplyExtraObjects(int index)
{
    bool extraOnState = index == extraObjectsOnIndex;
    SetGameObjects(extraObjectsOnAtIndex, extraOnState);

    bool extraOffState = index != extraObjectsOffIndex;
    SetGameObjects(extraObjectsOffAtIndex, extraOffState);
}

private void SetGameObjects(GameObject[] objects, bool state)
{
    if (objects == null)
    {
        return;
    }

    for (int i = 0; i < objects.Length; i++)
    {
        if (objects[i] != null)
        {
            if (objects[i].activeSelf != state)

```

```
{
  objects[i].SetActive(state);
}
}
}
}
```

Appendix 4 - GlobeTransformManager.cs

```

using UdonSharp;
using UnityEngine;
using UnityEngine.UI;
using VRC.SDKBase;

[UdonBehaviourSyncMode(BehaviourSyncMode.Manual)]
public class GlobeTransformManager : UdonSharpBehaviour
{
    [Header("Targets")]
    [Tooltip("The root pickup object. This usually has VRC_Pickup, Rigidbody, Collider and VRCObjectSync.")]
    public Transform pickupRoot;

    [Tooltip("The child object that will be scaled by the scale slider.")]
    public Transform scaleRoot;

    [Tooltip("The child pivot that will be rotated by the rotate slider.")]
    public Transform yawPivot;

    [Header("UI Sliders")]
    public Slider rotateSlider;
    public Slider scaleSlider;

    [Header("Yaw Settings")]
    public float minYaw = -180f;
    public float maxYaw = 180f;
    public Vector3 localYawAxis = Vector3.up;

    [Header("Scale Settings")]
    [Tooltip("Normal start scale. For your globe this should usually be 1.")]
    public float minScale = 1f;

    [Tooltip("Maximum enlarged scale.")]
    public float maxScale = 5f;

    [Header("Default Slider Values")]
    [Tooltip("0.5 means the rotate slider starts in the middle.")]
    [Range(0f, 1f)]
    public float defaultYaw01 = 0.5f;

    [Tooltip("0 means normal size when Min Scale is 1.")]
    [Range(0f, 1f)]
    public float defaultScale01 = 0f;

    [Header("Sync Settings")]
    public bool synced = true;

    [Tooltip("Delay after slider movement before sending network sync. Keeps it Quest-friendly.")]
    public float syncDelay = 0.35f;

    [UdonSynced] private float syncedYaw01 = 0.5f;
    [UdonSynced] private float syncedScale01 = 0f;

    private bool _ignoreSliderCallbacks;
    private bool _syncDirty;
    private float _nextSyncTime;

    private Quaternion _baseYawLocalRotation;
    private Vector3 _baseScaleRootLocalScale;

    private void Start()
    {
        if (yawPivot != null)
        {
            _baseYawLocalRotation = yawPivot.localRotation;
        }
    }
}

```

```

if (scaleRoot != null)
{
    _baseScaleRootLocalScale = scaleRoot.localScale;
}

ApplyAll(true);
}

private void Update()
{
    if (!syncd) return;
    if (!_syncDirty) return;

    if (Time.time >= _nextSyncTime)
    {
        CommitSyncedValues();
    }
}

public void OnRotateSliderChanged()
{
    if (_ignoreSliderCallbacks) return;
    if (rotateSlider == null) return;

    float value = Mathf.Clamp01(rotateSlider.value);

    ApplyYaw(value, false);

    if (syncd)
    {
        EnsureOwnership();
        syncedYaw01 = value;
        MarkDirtyForDelayedSync();
    }
}

public void OnScaleSliderChanged()
{
    if (_ignoreSliderCallbacks) return;
    if (scaleSlider == null) return;

    float value = Mathf.Clamp01(scaleSlider.value);

    ApplyScale(value, false);

    if (syncd)
    {
        EnsureOwnership();
        syncedScale01 = value;
        MarkDirtyForDelayedSync();
    }
}

public void ResetGlobeControls()
{
    EnsureOwnership();

    syncedYaw01 = Mathf.Clamp01(defaultYaw01);
    syncedScale01 = Mathf.Clamp01(defaultScale01);

    _syncDirty = false;

    ApplyAll(true);

    if (syncd)
    {
        RequestSerialization();
    }
}

```

```

}

public void RefreshFromManager()
{
    ApplyAll(true);
}

public override void OnDeserialization()
{
    if (!synced) return;

    ApplyAll(true);
}

public override void OnOwnershipTransferred(VRCPlayerApi player)
{
    ApplyAll(true);
}

private void ApplyAll(bool updateSliderUI)
{
    ApplyYaw(syncedYaw01, updateSliderUI);
    ApplyScale(syncedScale01, updateSliderUI);
}

private void ApplyYaw(float value01, bool updateSliderUI)
{
    value01 = Mathf.Clamp01(value01);

    if (yawPivot != null)
    {
        Vector3 axis = localYawAxis;

        if (axis == Vector3.zero)
        {
            axis = Vector3.up;
        }

        axis.Normalize();

        float angle = Mathf.Lerp(minYaw, maxYaw, value01);
        Quaternion rotationOffset = Quaternion.AngleAxis(angle, axis);

        yawPivot.localRotation = _baseYawLocalRotation * rotationOffset;
    }

    if (updateSliderUI && rotateSlider != null)
    {
        _ignoreSliderCallbacks = true;
        rotateSlider.value = value01;
        _ignoreSliderCallbacks = false;
    }
}

private void ApplyScale(float value01, bool updateSliderUI)
{
    value01 = Mathf.Clamp01(value01);

    if (scaleRoot != null)
    {
        float scaleValue = Mathf.Lerp(minScale, maxScale, value01);
        scaleRoot.localScale = _baseScaleRootLocalScale * scaleValue;
    }

    if (updateSliderUI && scaleSlider != null)
    {
        _ignoreSliderCallbacks = true;
        scaleSlider.value = value01;
    }
}

```

```

        _ignoreSliderCallbacks = false;
    }
}

private void MarkDirtyForDelayedSync()
{
    _syncDirty = true;
    _nextSyncTime = Time.time + syncDelay;
}

private void CommitSyncedValues()
{
    if (!synced) return;

    EnsureOwnership();

    _syncDirty = false;
    RequestSerialization();
}

private void EnsureOwnership()
{
    if (!synced) return;
    if (Networking.LocalPlayer == null) return;

    if (!Networking.IsOwner(gameObject))
    {
        Networking.SetOwner(Networking.LocalPlayer, gameObject);
    }
}
}

```

Appendix 5 - HybridObjectToggleGroup.cs

```

using UdonSharp;
using UnityEngine;
using UnityEngine.UI;

namespace StefanieInVR
{
    [UdonBehaviourSyncMode(BehaviourSyncMode.None)]
    public class HybridObjectToggleGroup : UdonSharpBehaviour
    {
        [Header("Required")]
        public HybridObjectToggleManager manager;

        [Tooltip("Index of this object group. Bird = 0.")]
        public int toggleIndex = 0;

        [Header("Controlled Objects")]
        [Tooltip("Objects that are active when the final/effective state is ON.")]
        public GameObject[] objectsOnWhenEnabled;

        [Tooltip("Objects that are active when the final/effective state is OFF. Usually leave empty.")]
        public GameObject[] objectsOnWhenDisabled;

        [Header("Global Toggle Sprite")]
        [Tooltip("Image on the GLOBAL button that should swap between ON and OFF sprite.")]
        public Image globalToggleImage;

        [Header("Local Toggle Sprite")]
        [Tooltip("Image on the LOCAL button that should swap between ON and OFF sprite.")]
        public Image localToggleImage;

        [Header("Sprites")]
        [Tooltip("Sprite shown when the state is ON.")]
        public Sprite onSprite;

        [Tooltip("Sprite shown when the state is OFF.")]
        public Sprite offSprite;

        [Header("Optional Local Override Indicator")]
        [Tooltip("Optional object that becomes active when this player has a local override.")]
        public GameObject localOverrideIndicator;

        [Header("Options")]
        public bool refreshOnStart = true;

        [Header("Debug")]
        public bool debugLogs = false;

        private void Start()
        {
            if (refreshOnStart)
            {
                SendCustomEventDelayedFrames(nameof(RefreshFromManager), 1);
            }
        }

        public void RefreshFromManager()
        {
            if (manager == null)
            {
                Debug.LogWarning("[HybridObjectToggleGroup] No manager assigned.");
                return;
            }

            bool globalState = manager.GetGlobalState(toggleIndex);
            bool effectiveState = manager.GetEffectiveState(toggleIndex);
            bool hasLocalOverride = manager.HasLocalOverride(toggleIndex);

```


VRChat Paper Tablet UI Product Handleiding - v1.6 - StefanieInVR

```
ApplyControlledObjects(effectiveState);
ApplyGlobalSprite(globalState);
ApplyLocalSprite(effectiveState);
ApplyLocalOverrideIndicator(hasLocalOverride);

if (debugLogs)
{
    Debug.Log("[HybridObjectToggleGroup] Refreshed index: " + toggleIndex + " Global: " + globalState + " Effective: " + effectiveState +
" Local override: " + hasLocalOverride);
}
}

private void ApplyControlledObjects(bool effectiveState)
{
    SetGameObjects(objectsOnWhenEnabled, effectiveState);
    SetGameObjects(objectsOnWhenDisabled, !effectiveState);
}

private void ApplyGlobalSprite(bool globalState)
{
    if (globalToggleImage == null)
    {
        return;
    }

    Sprite targetSprite = globalState ? onSprite : offSprite;

    if (targetSprite == null)
    {
        return;
    }

    if (globalToggleImage.sprite != targetSprite)
    {
        globalToggleImage.sprite = targetSprite;
    }
}

private void ApplyLocalSprite(bool effectiveState)
{
    if (localToggleImage == null)
    {
        return;
    }

    Sprite targetSprite = effectiveState ? onSprite : offSprite;

    if (targetSprite == null)
    {
        return;
    }

    if (localToggleImage.sprite != targetSprite)
    {
        localToggleImage.sprite = targetSprite;
    }
}

private void ApplyLocalOverrideIndicator(bool hasLocalOverride)
{
    if (localOverrideIndicator == null)
    {
        return;
    }

    if (localOverrideIndicator.activeSelf != hasLocalOverride)
    {

```

VRChat Paper Tablet UI Product Handleiding - v1.6 - StefanieInVR

```
        localOverrideIndicator.SetActive(hasLocalOverride);
    }
}

private void SetGameObjects(GameObject[] objects, bool state)
{
    if (objects == null)
    {
        return;
    }

    for (int i = 0; i < objects.Length; i++)
    {
        if (objects[i] != null)
        {
            if (objects[i].activeSelf != state)
            {
                objects[i].SetActive(state);
            }
        }
    }
}
}
```

Appendix 6 - HybridObjectToggleInput.cs

```

using UdonSharp;
using UnityEngine;

namespace StefanieInVR
{
    [UdonBehaviourSyncMode(BehaviourSyncMode.None)]
    public class HybridObjectToggleInput : UdonSharpBehaviour
    {
        [Header("Required")]
        public HybridObjectToggleManager manager;

        [Tooltip("Index of the object group. Bird = 0.")]
        public int toggleIndex = 0;

        [Header("Button Type")]
        [Tooltip("ON = this button changes the synced global state. OFF = this button changes only the local player view.")]
        public bool isGlobalButton = false;

        [Header("Optional Fixed Action")]
        [Tooltip("If true, this button always sets ON instead of toggling.")]
        public bool forceOn = false;

        [Tooltip("If true, this button always sets OFF instead of toggling.")]
        public bool forceOff = false;

        [Header("Local Only Option")]
        [Tooltip("Only for local buttons. If true, this clears the local override and returns to the global state.")]
        public bool clearLocalOverrideInstead = false;

        [Header("Debug")]
        public bool debugLogs = false;

        public override void Interact()
        {
            Press();
        }

        public void Press()
        {
            if (manager == null)
            {
                Debug.LogWarning("[HybridObjectToggleInput] No manager assigned.");
                return;
            }

            if (toggleIndex < 0 || toggleIndex > 30)
            {
                Debug.LogWarning("[HybridObjectToggleInput] Invalid toggleIndex: " + toggleIndex);
                return;
            }

            if (isGlobalButton)
            {
                PressGlobal();
            }
            else
            {
                PressLocal();
            }
        }

        public void PressGlobal()
        {
            if (manager == null)
            {
                Debug.LogWarning("[HybridObjectToggleInput] PressGlobal failed. No manager assigned.");
            }
        }
    }
}

```

```

        return;
    }

    if (forceOn)
    {
        manager.SetGlobalOn(toggleIndex);
    }
    else if (forceOff)
    {
        manager.SetGlobalOff(toggleIndex);
    }
    else
    {
        manager.ToggleGlobal(toggleIndex);
    }

    if (debugLogs)
    {
        Debug.Log("[HybridObjectToggleInput] Global button pressed. Index: " + toggleIndex);
    }
}

public void PressLocal()
{
    if (manager == null)
    {
        Debug.LogWarning("[HybridObjectToggleInput] PressLocal failed. No manager assigned.");
        return;
    }

    if (clearLocalOverrideInstead)
    {
        manager.ClearLocalOverride(toggleIndex);
    }
    else
    {
        manager.ToggleLocal(toggleIndex);
    }

    if (debugLogs)
    {
        Debug.Log("[HybridObjectToggleInput] Local button pressed. Index: " + toggleIndex);
    }
}
}
}

```

Appendix 7 - HybridObjectToggleManager.cs

```

using UdonSharp;
using UnityEngine;
using VRC.SDKBase;

namespace StefanieInVR
{
    [UdonBehaviourSyncMode(BehaviourSyncMode.Manual)]
    public class HybridObjectToggleManager : UdonSharpBehaviour
    {
        [Header("Default Global States")]
        [Tooltip("Index 0 = Bird. True = starts ON, False = starts OFF.")]
        public bool[] defaultGlobalStates;

        [Header("Toggle Groups")]
        [Tooltip("One group per object toggle. Example: Bird group = index 0.")]
        public HybridObjectToggleGroup[] groups;

        [Header("Debug")]
        public bool debugLogs = false;

        [UdonSynced]
        private int globalStateBits = 0;

        [UdonSynced]
        private int globalRevision = 0;

        [UdonSynced]
        private bool hasInitializedNetworkState = false;

        private int localOverrideBits = 0;
        private int localValueBits = 0;
        private int lastAppliedGlobalRevision = -1;

        private bool hasStarted = false;

        private void Start()
        {
            hasStarted = true;

            if (!hasInitializedNetworkState)
            {
                globalStateBits = BuildDefaultBits();

                if (Networking.LocalPlayer != null && Networking.IsOwner(gameObject))
                {
                    hasInitializedNetworkState = true;
                    RequestSerialization();

                    if (debugLogs)
                    {
                        Debug.Log("[HybridObjectToggleManager] Initialized default global states.");
                    }
                }
            }

            lastAppliedGlobalRevision = globalRevision;

            SendCustomEventDelayedFrames(nameof(RefreshAll), 1);
        }

        public override void OnDeserialization()
        {
            ApplyGlobalRevisionIfNeeded();
            RefreshAll();

            if (debugLogs)

```

VRChat Paper Tablet UI Product Handleiding - v1.6 - StefanieInVR

```
{
    Debug.Log("[HybridObjectToggleManager] Deserialized. Revision: " + globalRevision);
}

public void ToggleGlobal(int toggleIndex)
{
    if (!IsValidIndex(toggleIndex))
    {
        Debug.LogWarning("[HybridObjectToggleManager] ToggleGlobal failed. Invalid index: " + toggleIndex);
        return;
    }

    TakeOwnershipIfNeeded();

    bool currentState = GetGlobalState(toggleIndex);
    SetGlobalStateInternal(toggleIndex, !currentState);
}

public void SetGlobalOn(int toggleIndex)
{
    if (!IsValidIndex(toggleIndex))
    {
        Debug.LogWarning("[HybridObjectToggleManager] SetGlobalOn failed. Invalid index: " + toggleIndex);
        return;
    }

    TakeOwnershipIfNeeded();
    SetGlobalStateInternal(toggleIndex, true);
}

public void SetGlobalOff(int toggleIndex)
{
    if (!IsValidIndex(toggleIndex))
    {
        Debug.LogWarning("[HybridObjectToggleManager] SetGlobalOff failed. Invalid index: " + toggleIndex);
        return;
    }

    TakeOwnershipIfNeeded();
    SetGlobalStateInternal(toggleIndex, false);
}

public void ToggleLocal(int toggleIndex)
{
    if (!IsValidIndex(toggleIndex))
    {
        Debug.LogWarning("[HybridObjectToggleManager] ToggleLocal failed. Invalid index: " + toggleIndex);
        return;
    }

    ApplyGlobalRevisionIfNeeded();

    bool currentEffectiveState = GetEffectiveState(toggleIndex);
    bool newLocalState = !currentEffectiveState;

    localOverrideBits = SetBit(localOverrideBits, toggleIndex, true);
    localValueBits = SetBit(localValueBits, toggleIndex, newLocalState);

    RefreshAll();

    if (debugLogs)
    {
        Debug.Log("[HybridObjectToggleManager] Local toggle changed. Index: " + toggleIndex + " State: " + newLocalState);
    }
}

public void ClearLocalOverride(int toggleIndex)
```

VRChat Paper Tablet UI Product Handleiding - v1.6 - StefanieInVR

```
{
    if (!IsValidIndex(toggleIndex))
    {
        Debug.LogWarning("[HybridObjectToggleManager] ClearLocalOverride failed. Invalid index: " + toggleIndex);
        return;
    }

    localOverrideBits = SetBit(localOverrideBits, toggleIndex, false);
    RefreshAll();

    if (debugLogs)
    {
        Debug.Log("[HybridObjectToggleManager] Local override cleared. Index: " + toggleIndex);
    }
}

public void ClearAllLocalOverrides()
{
    localOverrideBits = 0;
    localValueBits = 0;

    RefreshAll();

    if (debugLogs)
    {
        Debug.Log("[HybridObjectToggleManager] All local overrides cleared.");
    }
}

public bool GetGlobalState(int toggleIndex)
{
    if (!IsValidIndex(toggleIndex))
    {
        return false;
    }

    return GetBit(globalStateBits, toggleIndex);
}

public bool GetEffectiveState(int toggleIndex)
{
    if (!IsValidIndex(toggleIndex))
    {
        return false;
    }

    ApplyGlobalRevisionIfNeeded();

    bool hasLocalOverride = GetBit(localOverrideBits, toggleIndex);

    if (hasLocalOverride)
    {
        return GetBit(localValueBits, toggleIndex);
    }

    return GetGlobalState(toggleIndex);
}

public bool HasLocalOverride(int toggleIndex)
{
    if (!IsValidIndex(toggleIndex))
    {
        return false;
    }

    ApplyGlobalRevisionIfNeeded();
    return GetBit(localOverrideBits, toggleIndex);
}
```

```

public int GetGlobalRevision()
{
    return globalRevision;
}

public void RefreshAll()
{
    ApplyGlobalRevisionIfNeeded();

    if (groups == null)
    {
        return;
    }

    for (int i = 0; i < groups.Length; i++)
    {
        if (groups[i] != null)
        {
            groups[i].RefreshFromManager();
        }
    }
}

private void SetGlobalStateInternal(int toggleIndex, bool newState)
{
    globalStateBits = SetBit(globalStateBits, toggleIndex, newState);

    globalRevision++;

    hasInitializedNetworkState = true;

    localOverrideBits = 0;
    localValueBits = 0;
    lastAppliedGlobalRevision = globalRevision;

    RequestSerialization();
    RefreshAll();

    if (debugLogs)
    {
        Debug.Log("[HybridObjectToggleManager] Global state changed. Index: " + toggleIndex + " State: " + newState + " Revision: " +
globalRevision);
    }
}

private void ApplyGlobalRevisionIfNeeded()
{
    if (lastAppliedGlobalRevision != globalRevision)
    {
        localOverrideBits = 0;
        localValueBits = 0;
        lastAppliedGlobalRevision = globalRevision;

        if (debugLogs && hasStarted)
        {
            Debug.Log("[HybridObjectToggleManager] Global revision changed. Local overrides cleared.");
        }
    }
}

private void TakeOwnershipIfNeeded()
{
    if (Networking.LocalPlayer == null)
    {
        return;
    }
}

```


VRChat Paper Tablet UI Product Handleiding - v1.6 - StefanieInVR

```
if (!Networking.IsOwner(gameObject))
{
    Networking.SetOwner(Networking.LocalPlayer, gameObject);
}

private int BuildDefaultBits()
{
    int bits = 0;

    if (defaultGlobalStates == null)
    {
        return bits;
    }

    int max = defaultGlobalStates.Length;

    if (max > 31)
    {
        max = 31;
    }

    for (int i = 0; i < max; i++)
    {
        bits = SetBit(bits, i, defaultGlobalStates[i]);
    }

    return bits;
}

private bool IsValidIndex(int toggleIndex)
{
    if (toggleIndex < 0)
    {
        return false;
    }

    if (toggleIndex > 30)
    {
        return false;
    }

    return true;
}

private bool GetBit(int bits, int index)
{
    int mask = 1 << index;
    return (bits & mask) != 0;
}

private int SetBit(int bits, int index, bool value)
{
    int mask = 1 << index;

    if (value)
    {
        return bits | mask;
    }

    return bits & ~mask;
}
}
```

Appendix 8 - PaperTabletTabButton.cs

```
using UdonSharp;
using UnityEngine;

public class PaperTabletTabButton : UdonSharpBehaviour
{
    [Header("Manager")]
    public PaperTabletTabManager tabManager;

    [Header("Welke tab opent deze knop?")]
    public int tabIndex = 0;

    public void OnButtonPressed()
    {
        if (tabManager == null) return;

        tabManager.SelectTab(tabIndex);
    }
}
```

Appendix 9 - PaperTabletTabManager.cs

```

using UdonSharp;
using UnityEngine;
using UnityEngine.UI;
using VRC.SDKBase;

[UdonBehaviourSyncMode(BehaviourSyncMode.Manual)]
public class PaperTabletTabManager : UdonSharpBehaviour
{
    [Header("1. Tab Panels")]
    [Tooltip("Zelfde volgorde als je tabs. Element 0 = tab 0, element 1 = tab 1, enz.")]
    public GameObject[] tabPanels;

    [Header("2. Tab Buttons")]
    [Tooltip("Zelfde volgorde als je tabs.")]
    public Button[] tabButtons;

    [Header("3. Button Sprites")]
    public Sprite[] tabOffSprites;
    public Sprite[] tabOffHighSprites;
    public Sprite[] tabOnSprites;
    public Sprite[] tabOnHighSprites;

    [Header("4. Tab States")]
    [Tooltip("Sleep hier de PaperTabletTabState scripts in. Elke TabState zegt welke extra objecten zichtbaar zijn op die tab.")]
    public PaperTabletTabState[] tabStates;

    [Header("5. VIP Access Filter")]
    [Tooltip("Sleep hier je VipAccessManager in. Die bepaalt lokaal of deze speler tabs mag bedienen en filtert VIP/locked inhoud.")]
    public VipAccessManager vipAccessManager;

    [Header("6. Start Tab")]
    public int defaultTabIndex = 0;

    [UdonSynced] private int selectedTabIndex = 0;

    private bool hasStarted = false;

    private bool pendingSerialization = false;
    private int serializationTries = 0;

    private void Start()
    {
        if (Networking.IsOwner(gameObject))
        {
            selectedTabIndex = ClampTabIndex(defaultTabIndex);
            RequestSerialization();
        }

        hasStarted = true;
        RefreshAll();
    }

    public override void OnDeserialization()
    {
        if (!hasStarted) return;

        RefreshAll();
    }

    public void SelectTab(int tabIndex)
    {
        tabIndex = ClampTabIndex(tabIndex);

        if (vipAccessManager != null)
        {
            if (!vipAccessManager.CanLocalPlayerSelectTab(tabIndex, selectedTabIndex))

```

```

    {
        return;
    }
}

if (selectedTabIndex == tabIndex)
{
    RefreshAll();
    return;
}

int previousTabIndex = selectedTabIndex;

TakeOwnershipIfNeeded();

selectedTabIndex = tabIndex;

RefreshAllForTabChange(previousTabIndex, selectedTabIndex);

RequestSerializationSafe();
}

private void TakeOwnershipIfNeeded()
{
    if (Networking.IsOwner(gameObject)) return;
    if (Networking.LocalPlayer == null) return;

    Networking.SetOwner(Networking.LocalPlayer, gameObject);
}

private void RequestSerializationSafe()
{
    pendingSerialization = true;
    serializationTries = 0;
    SendCustomEventDelayedFrames(nameof(DelayedRequestSerialization), 1);
}

public void DelayedRequestSerialization()
{
    if (!pendingSerialization) return;

    if (!Networking.IsOwner(gameObject))
    {
        serializationTries++;

        if (serializationTries < 10)
        {
            SendCustomEventDelayedFrames(nameof(DelayedRequestSerialization), 1);
        }

        return;
    }

    pendingSerialization = false;
    RequestSerialization();
}

public int GetSelectedTabIndex()
{
    return selectedTabIndex;
}

public bool IsTabSelected(int tabIndex)
{
    return selectedTabIndex == tabIndex;
}

public void RefreshAll()

```

VRChat Paper Tablet UI Product Handleiding - v1.6 - StefanieInVR

```
{
    if (vipAccessManager != null && vipAccessManager.ShouldHideTabletContentForLocalPlayer(selectedTabIndex))
    {
        HideAllPanelsAndTabStateObjects();
        RefreshButtonVisuals();
        ApplyVipAccessFilter();
        return;
    }

    RefreshPanels();
    RefreshAllExtraObjectsForCurrentTab();
    RefreshButtonVisuals();
    ApplyVipAccessFilter();
}

private void RefreshAllForTabChange(int oldTabIndex, int newTabIndex)
{
    if (vipAccessManager != null && vipAccessManager.ShouldHideTabletContentForLocalPlayer(newTabIndex))
    {
        HideAllPanelsAndTabStateObjects();
        RefreshButtonVisuals();
        ApplyVipAccessFilter();
        return;
    }

    RefreshPanels();
    RefreshExtraObjectsTransition(oldTabIndex, newTabIndex);
    RefreshButtonVisuals();
    ApplyVipAccessFilter();
}

public void HideAllPanelsAndTabStateObjects()
{
    if (tabPanels != null)
    {
        for (int i = 0; i < tabPanels.Length; i++)
        {
            SetActiveIfNeeded(tabPanels[i], false);
        }
    }

    if (tabStates != null)
    {
        for (int s = 0; s < tabStates.Length; s++)
        {
            PaperTabletTabState state = tabStates[s];

            if (state == null) continue;
            if (state.objectsVisibleOnThisTab == null) continue;

            for (int i = 0; i < state.objectsVisibleOnThisTab.Length; i++)
            {
                SetActiveIfNeeded(state.objectsVisibleOnThisTab[i], false);
            }
        }
    }
}

private void RefreshPanels()
{
    if (tabPanels == null) return;

    for (int i = 0; i < tabPanels.Length; i++)
    {
        SetActiveIfNeeded(tabPanels[i], i == selectedTabIndex);
    }
}
```

VRChat Paper Tablet UI Product Handleiding - v1.6 - StefanieInVR

```
private void RefreshExtraObjectsTransition(int oldTabIndex, int newTabIndex)
```

```
{
    if (tabStates == null) return;

    for (int s = 0; s < tabStates.Length; s++)
    {
        PaperTabletTabState state = tabStates[s];

        if (state == null) continue;
        if (state.tabIndex != oldTabIndex) continue;
        if (state.objectsVisibleOnThisTab == null) continue;

        for (int i = 0; i < state.objectsVisibleOnThisTab.Length; i++)
        {
            GameObject obj = state.objectsVisibleOnThisTab[i];

            if (obj == null) continue;

            if (!ShouldObjectBeVisibleOnTab(obj, newTabIndex))
            {
                SetActiveIfNeeded(obj, false);
            }
        }
    }

    for (int s = 0; s < tabStates.Length; s++)
    {
        PaperTabletTabState state = tabStates[s];

        if (state == null) continue;
        if (state.tabIndex != newTabIndex) continue;
        if (state.objectsVisibleOnThisTab == null) continue;

        for (int i = 0; i < state.objectsVisibleOnThisTab.Length; i++)
        {
            SetActiveIfNeeded(state.objectsVisibleOnThisTab[i], true);
        }
    }
}
```

```
private void RefreshAllExtraObjectsForCurrentTab()
```

```
{
    if (tabStates == null) return;

    for (int s = 0; s < tabStates.Length; s++)
    {
        PaperTabletTabState state = tabStates[s];

        if (state == null) continue;
        if (state.objectsVisibleOnThisTab == null) continue;

        for (int i = 0; i < state.objectsVisibleOnThisTab.Length; i++)
        {
            GameObject obj = state.objectsVisibleOnThisTab[i];

            if (obj == null) continue;

            bool shouldBeVisible = ShouldObjectBeVisibleOnTab(obj, selectedTabIndex);
            SetActiveIfNeeded(obj, shouldBeVisible);
        }
    }
}
```

```
private bool ShouldObjectBeVisibleOnTab(GameObject obj, int tabIndex)
```

```
{
    if (obj == null) return false;
    if (tabStates == null) return false;
```

VRChat Paper Tablet UI Product Handleiding - v1.6 - StefanieInVR

```
for (int s = 0; s < tabStates.Length; s++)
{
    PaperTabletTabState state = tabStates[s];

    if (state == null) continue;
    if (state.tabIndex != tabIndex) continue;
    if (state.objectsVisibleOnThisTab == null) continue;

    for (int i = 0; i < state.objectsVisibleOnThisTab.Length; i++)
    {
        if (state.objectsVisibleOnThisTab[i] == obj)
        {
            return true;
        }
    }
}

return false;
}

private void ApplyVipAccessFilter()
{
    if (vipAccessManager != null)
    {
        vipAccessManager.ApplyAccessForCurrentTab(selectedTabIndex);
    }
}

private void RefreshButtonVisuals()
{
    if (tabButtons == null) return;

    for (int i = 0; i < tabButtons.Length; i++)
    {
        Button button = tabButtons[i];

        if (button == null) continue;
        if (button.image == null) continue;

        bool isActive = i == selectedTabIndex;

        Sprite normalSprite = GetSprite(isActive ? tabOnSprites : tabOffSprites, i);
        Sprite highSprite = GetSprite(isActive ? tabOnHighSprites : tabOffHighSprites, i);

        if (normalSprite != null)
        {
            button.image.sprite = normalSprite;
        }

        SpriteState spriteState = button.spriteState;

        if (highSprite != null)
        {
            spriteState.highlightedSprite = highSprite;
            spriteState.pressedSprite = highSprite;
            spriteState.selectedSprite = highSprite;
        }

        button.spriteState = spriteState;
    }
}

private Sprite GetSprite(Sprite[] sprites, int index)
{
    if (sprites == null) return null;
    if (index < 0 || index >= sprites.Length) return null;

    return sprites[index];
}
```

```
}

private void SetActiveIfNeeded(GameObject target, bool active)
{
    if (target == null) return;

    if (target.activeSelf != active)
    {
        target.SetActive(active);
    }
}

private int ClampTabIndex(int tabIndex)
{
    if (tabPanels == null || tabPanels.Length == 0)
    {
        return 0;
    }

    if (tabIndex < 0) return 0;
    if (tabIndex >= tabPanels.Length) return tabPanels.Length - 1;

    return tabIndex;
}
}
```


Appendix 10 - PaperTabletTabState.cs

```
using UdonSharp;
using UnityEngine;

[UdonBehaviourSyncMode(BehaviourSyncMode.None)]
public class PaperTabletTabState : UdonSharpBehaviour
{
    [Header("1. Tab")]
    [Tooltip("Bij welke tab hoort deze lijst? 0 = Video, 3 = Playlists, 6 = VIP.")]
    public int tabIndex = 0;

    [Header("2. Objects Visible On This Tab")]
    [Tooltip("Sleep hier alle extra objecten in die zichtbaar moeten zijn wanneer deze tab actief is.")]
    public GameObject[] objectsVisibleOnThisTab;
}
```

Appendix 11 - PaperTabletToggleButton.cs

```
using UdonSharp;
using UnityEngine;

namespace StefanieInVR.PaperTablet
{

public class PaperTabletToggleButton : UdonSharpBehaviour
{
    [Header("Manager")]
    public PaperTabletToggleManager toggleManager;

    [Header("Welke toggle bestuur ik?")]
    public int toggleIndex = 0;

    public void OnButtonPressed()
    {
        if (toggleManager == null) return;

        toggleManager.ToggleFromButton(toggleIndex);
    }
}
```

Appendix 12 - PaperTabletToggleManager.cs

```

using UdonSharp;
using UnityEngine;
using VRC.SDKBase;

namespace StefanieInVR.PaperTablet
{
    [UdonBehaviourSyncMode(BehaviourSyncMode.Manual)]
    public class PaperTabletToggleManager : UdonSharpBehaviour
    {
        [Header("Alle visuals die moeten verversen")]
        public PaperTabletToggleVisual[] visuals;

        [Header("Startstatus per toggle - max 32")]
        public bool[] startStates;

        [Header("Sync per toggle - true = global, false = local")]
        [Tooltip("True = gesynct voor iedereen. False = alleen lokaal voor deze speler. Als deze lijst te kort is, wordt die toggle automatisch als global behandeld.")]
        public bool[] syncToggles;

        [UdonSynced] private int stateBits = 0;

        private int localStateBits = 0;
        private bool hasStarted = false;

        private bool pendingSerialization = false;
        private int serializationTries = 0;

        private void Start()
        {
            InitializeLocalStates();

            if (Networking.IsOwner(gameObject))
            {
                InitializeGlobalStates();
                RequestSerialization();
            }

            hasStarted = true;
            RefreshAllVisuals();
        }

        public override void OnDeserialization()
        {
            if (!hasStarted) return;

            RefreshAllVisuals();
        }

        private void InitializeGlobalStates()
        {
            stateBits = 0;

            if (startStates == null) return;

            int count = startStates.Length;

            if (count > 32)
            {
                count = 32;
            }

            for (int i = 0; i < count; i++)
            {
                if (!IsToggleGlobal(i)) continue;
            }
        }
    }
}

```

```

        if (startStates[i])
        {
            stateBits |= (1 << i);
        }
    }
}

private void InitializeLocalStates()
{
    localStateBits = 0;

    if (startStates == null) return;

    int count = startStates.Length;

    if (count > 32)
    {
        count = 32;
    }

    for (int i = 0; i < count; i++)
    {
        if (IsToggleGlobal(i)) continue;

        if (startStates[i])
        {
            localStateBits |= (1 << i);
        }
    }
}

public bool GetToggleState(int toggleIndex)
{
    if (toggleIndex < 0 || toggleIndex > 31)
    {
        return false;
    }

    if (IsToggleGlobal(toggleIndex))
    {
        return (stateBits & (1 << toggleIndex)) != 0;
    }

    return (localStateBits & (1 << toggleIndex)) != 0;
}

public bool IsToggleGlobal(int toggleIndex)
{
    if (toggleIndex < 0 || toggleIndex > 31)
    {
        return true;
    }

    if (syncToggles == null)
    {
        return true;
    }

    if (toggleIndex >= syncToggles.Length)
    {
        return true;
    }

    return syncToggles[toggleIndex];
}

public void ToggleFromButton(int toggleIndex)

```

```

{
    if (toggleIndex < 0 || toggleIndex > 31)
    {
        return;
    }

    if (IsToggleGlobal(toggleIndex))
    {
        ToggleGlobal(toggleIndex);
    }
    else
    {
        ToggleLocal(toggleIndex);
    }
}

private void ToggleGlobal(int toggleIndex)
{
    TakeOwnershipIfNeeded();

    if (!Networking.IsOwner(gameObject)) return;

    stateBits ^= (1 << toggleIndex);

    RefreshAllVisuals();
    RequestSerializationSafe();
}

private void ToggleLocal(int toggleIndex)
{
    localStateBits ^= (1 << toggleIndex);

    RefreshAllVisuals();
}

public void SetToggleState(int toggleIndex, bool state)
{
    if (toggleIndex < 0 || toggleIndex > 31)
    {
        return;
    }

    if (IsToggleGlobal(toggleIndex))
    {
        SetGlobalToggleState(toggleIndex, state);
    }
    else
    {
        SetLocalToggleState(toggleIndex, state);
    }
}

private void SetGlobalToggleState(int toggleIndex, bool state)
{
    TakeOwnershipIfNeeded();

    if (!Networking.IsOwner(gameObject)) return;

    bool oldState = GetToggleState(toggleIndex);

    if (oldState == state)
    {
        RefreshAllVisuals();
        return;
    }

    if (state)
    {

```

```

        stateBits |= (1 << toggleIndex);
    }
    else
    {
        stateBits &= ~(1 << toggleIndex);
    }

    RefreshAllVisuals();
    RequestSerializationSafe();
}

private void SetLocalToggleState(int toggleIndex, bool state)
{
    bool oldState = GetToggleState(toggleIndex);

    if (oldState == state)
    {
        RefreshAllVisuals();
        return;
    }

    if (state)
    {
        localStateBits |= (1 << toggleIndex);
    }
    else
    {
        localStateBits &= ~(1 << toggleIndex);
    }

    RefreshAllVisuals();
}

private void TakeOwnershipIfNeeded()
{
    if (Networking.IsOwner(gameObject)) return;
    if (Networking.LocalPlayer == null) return;

    Networking.SetOwner(Networking.LocalPlayer, gameObject);
}

private void RequestSerializationSafe()
{
    pendingSerialization = true;
    serializationTries = 0;
    SendCustomEventDelayedFrames(nameof(DelayedRequestSerialization), 1);
}

public void DelayedRequestSerialization()
{
    if (!pendingSerialization) return;

    if (!Networking.IsOwner(gameObject))
    {
        serializationTries++;

        if (serializationTries < 10)
        {
            SendCustomEventDelayedFrames(nameof(DelayedRequestSerialization), 1);
        }

        return;
    }

    pendingSerialization = false;
    RequestSerialization();
}

```

VRChat Paper Tablet UI Product Handleiding - v1.6 - StefanieInVR

```
public void RefreshAllVisuals()
{
    if (visuals == null) return;

    for (int i = 0; i < visuals.Length; i++)
    {
        if (visuals[i] != null)
        {
            visuals[i].RefreshVisual();
        }
    }
}
```

Appendix 13 - PaperTabletToggleVisual.cs

```

using UdonSharp;
using UnityEngine;
using UnityEngine.UI;
using TMPro;

namespace StefanieInVR.PaperTablet
{
    public class PaperTabletToggleVisual : UdonSharpBehaviour
    {
        [Header("Manager + index")]
        public PaperTabletToggleManager toggleManager;
        public int toggleIndex = 0;

        [Header("Objecten die aan/uit moeten")]
        public GameObject[] targetObjects;

        [Header("Button - optioneel")]
        public Button targetButton;

        [Header("Sprites - OFF")]
        public Sprite spriteOff;

        [Header("Sprites - OFF Hover/Highlight")]
        public Sprite spriteOffHigh;

        [Header("Sprites - ON")]
        public Sprite spriteOn;

        [Header("Sprites - ON Hover/Highlight")]
        public Sprite spriteOnHigh;

        [Header("Tekst UI - optioneel")]
        public TextMeshProUGUI[] labelTextsUI;

        [Header("Tekst World - optioneel")]
        public TextMeshPro[] labelTextsWorld;

        [Header("Tekst")]
        public string textOff = "Off";
        public string textOn = "On";

        [Header("Tekstkleur - optioneel")]
        public bool useTextColors = false;
        public Color textColorOff = Color.white;
        public Color textColorOn = Color.green;

        private void Start()
        {
            RefreshVisual();
        }

        public void RefreshVisual()
        {
            if (toggleManager == null) return;

            bool isOn = toggleManager.GetToggleState(toggleIndex);

            RefreshTargetObjects(isOn);
            RefreshButtonSprites(isOn);
            RefreshTexts(isOn);
        }

        private void RefreshTargetObjects(bool isOn)
        {
            if (targetObjects == null) return;

```



```

for (int i = 0; i < targetObjects.Length; i++)
{
    if (targetObjects[i] != null)
    {
        targetObjects[i].SetActive(isOn);
    }
}

private void RefreshButtonSprites(bool.isOn)
{
    if (targetButton == null) return;
    if (targetButton.image == null) return;

    Sprite normalSprite = isOn ? spriteOn : spriteOff;
    Sprite highSprite = isOn ? spriteOnHigh : spriteOffHigh;

    if (normalSprite != null)
    {
        targetButton.image.sprite = normalSprite;
    }

    SpriteState spriteState = targetButton.spriteState;

    if (highSprite != null)
    {
        spriteState.highlightedSprite = highSprite;
        spriteState.pressedSprite = highSprite;
        spriteState.selectedSprite = highSprite;
    }

    targetButton.spriteState = spriteState;
}

private void RefreshTexts(bool.isOn)
{
    if (labelTextsUI != null)
    {
        for (int i = 0; i < labelTextsUI.Length; i++)
        {
            if (labelTextsUI[i] != null)
            {
                labelTextsUI[i].text = isOn ? textOn : textOff;

                if (useTextColors)
                {
                    labelTextsUI[i].color = isOn ? textColorOn : textColorOff;
                }
            }
        }
    }

    if (labelTextsWorld != null)
    {
        for (int i = 0; i < labelTextsWorld.Length; i++)
        {
            if (labelTextsWorld[i] != null)
            {
                labelTextsWorld[i].text = isOn ? textOn : textOff;

                if (useTextColors)
                {
                    labelTextsWorld[i].color = isOn ? textColorOn : textColorOff;
                }
            }
        }
    }
}

```

```
}  
}  
}
```

Appendix 14 - ProjectorScreenToggle.cs

```

using UdonSharp;
using UnityEngine;
using UnityEngine.UI;
using VRC.SDKBase;

namespace StefanieInVR.PaperTablet
{
    [UdonBehaviourSyncMode(BehaviourSyncMode.Manual)]
    public class ProjectorScreenToggle : UdonSharpBehaviour
    {
        [Header("Screen Blendshape")]
        [Tooltip("The Skinned Mesh Renderer that has the projector screen blendshape.")]
        public SkinnedMeshRenderer targetRenderer;

        [Tooltip("Blendshape index. Usually 0 if this mesh has only one blendshape.")]
        public int blendShapeIndex = 0;

        [Header("Open / Closed Values")]
        [Range(0f, 100f)]
        public float closedWeight = 0f;

        [Range(0f, 100f)]
        public float openWeight = 100f;

        [Header("Movement")]
        public bool smooth = true;

        [Tooltip("How fast the screen moves. 50 is calm. 100 is faster.")]
        public float smoothSpeed = 50f;

        [Header("Start State")]
        public bool startOpen = true;

        [Header("Optional Button Visuals")]
        public Image buttonImage;
        public Button uiButton;

        [Header("Normal Sprites")]
        public Sprite normalClosed;
        public Sprite normalOpen;

        [Header("Highlighted Sprites")]
        public Sprite highlightedClosed;
        public Sprite highlightedOpen;

        [Header("Pressed Sprites")]
        public Sprite pressedClosed;
        public Sprite pressedOpen;

        [Header("Optional Owner Lock")]
        public bool lockButtonToOwner = false;

        [UdonSynced]
        private bool isOpen;

        private float currentWeight;
        private float targetWeight;
        private bool isAnimating;

        private bool pendingSerialization;
        private int serializationTries;
        private bool hasStarted;

        private void Start()
        {
            if (Networking.IsOwner(gameObject))

```

```

{
    isOpen = startOpen;
    RequestSerialization();
}

hasStarted = true;
ApplyStateImmediate();
UpdateButtonVisual();
UpdateButtonLock();
}

private void Update()
{
    if (!isAnimating)
    {
        return;
    }

    if (targetRenderer == null)
    {
        isAnimating = false;
        return;
    }

    currentWeight = Mathf.MoveTowards(
        currentWeight,
        targetWeight,
        smoothSpeed * Time.deltaTime
    );

    targetRenderer.SetBlendShapeWeight(blendShapeIndex, currentWeight);

    if (Mathf.Approximately(currentWeight, targetWeight))
    {
        isAnimating = false;
    }
}

public void ToggleScreen()
{
    SetScreenOpen(!isOpen);
}

public void OpenScreen()
{
    SetScreenOpen(true);
}

public void CloseScreen()
{
    SetScreenOpen(false);
}

public void SetScreenOpen(bool open)
{
    TakeOwnershipIfNeeded();

    isOpen = open;

    ApplyTargetFromState();
    UpdateButtonVisual();
    UpdateButtonLock();

    RequestSerializationSafe();
}

public bool IsOpen()
{

```

```

    return isOpen;
}

public override void OnDeserialization()
{
    if (!hasStarted)
    {
        return;
    }

    ApplyTargetFromState();
    UpdateButtonVisual();
    UpdateButtonLock();
}

public override void Interact()
{
    ToggleScreen();
}

public override void OnOwnershipTransferred(VRCPlayerApi player)
{
    UpdateButtonLock();

    if (pendingSerialization && Networking.IsOwner(gameObject))
    {
        SendCustomEventDelayedFrames(nameof(DelayedRequestSerialization), 1);
    }
}

private void ApplyStateImmediate()
{
    targetWeight = isOpen ? openWeight : closedWeight;
    ApplyWeightImmediate(targetWeight);
}

private void ApplyTargetFromState()
{
    targetWeight = isOpen ? openWeight : closedWeight;

    if (smooth)
    {
        if (targetRenderer != null)
        {
            currentWeight = targetRenderer.GetBlendShapeWeight(blendShapeIndex);
        }

        isAnimating = true;
    }
    else
    {
        ApplyWeightImmediate(targetWeight);
    }
}

private void ApplyWeightImmediate(float weight)
{
    if (targetRenderer == null)
    {
        return;
    }

    currentWeight = weight;
    targetWeight = weight;
    isAnimating = false;

    targetRenderer.SetBlendShapeWeight(blendShapeIndex, weight);
}

```

```

private void UpdateButtonVisual()
{
    Sprite normalSprite = isOpen ? normalOpen : normalClosed;
    Sprite highlightedSprite = isOpen ? highlightedOpen : highlightedClosed;
    Sprite pressedSprite = isOpen ? pressedOpen : pressedClosed;

    if (highlightedSprite == null)
    {
        highlightedSprite = normalSprite;
    }

    if (pressedSprite == null)
    {
        pressedSprite = highlightedSprite;
    }

    if (buttonImage != null && normalSprite != null)
    {
        buttonImage.sprite = normalSprite;
    }

    if (uiButton != null)
    {
        if (buttonImage != null && uiButton.targetGraphic != buttonImage)
        {
            uiButton.targetGraphic = buttonImage;
        }

        SpriteState state = uiButton.spriteState;

        state.highlightedSprite = highlightedSprite;
        state.pressedSprite = pressedSprite;
        state.selectedSprite = null;
        state.disabledSprite = normalSprite;

        uiButton.spriteState = state;
    }
}

private void UpdateButtonLock()
{
    if (!lockButtonToOwner)
    {
        return;
    }

    if (uiButton == null)
    {
        return;
    }

    uiButton.interactable = Networking.IsOwner(gameObject);
}

private void TakeOwnershipIfNeeded()
{
    if (Networking.IsOwner(gameObject))
    {
        return;
    }

    if (Networking.LocalPlayer == null)
    {
        return;
    }

    Networking.SetOwner(Networking.LocalPlayer, gameObject);
}

```

```
}

private void RequestSerializationSafe()
{
    pendingSerialization = true;
    serializationTries = 0;
    SendCustomEventDelayedFrames(nameof(DelayedRequestSerialization), 1);
}

public void DelayedRequestSerialization()
{
    if (!pendingSerialization)
    {
        return;
    }

    if (!Networking.IsOwner(gameObject))
    {
        serializationTries++;

        if (serializationTries < 10)
        {
            SendCustomEventDelayedFrames(nameof(DelayedRequestSerialization), 1);
        }

        return;
    }

    pendingSerialization = false;
    RequestSerialization();
}
}
```

Appendix 15 - ResetObjectManager.cs

```

using UdonSharp;
using UnityEngine;

namespace StefanieInVR.PaperTablet
{

public class ResetObjectManager : UdonSharpBehaviour
{
    [Header("Objects To Reset")]
    [Tooltip("Drag all objects here that have a ResettableObject script.")]
    public ResettableObject[] resettableObjects;

    public void ResetEverything()
    {
        if (resettableObjects == null)
        {
            return;
        }

        for (int i = 0; i < resettableObjects.Length; i++)
        {
            ResettableObject resetObject = resettableObjects[i];

            if (resetObject != null)
            {
                resetObject.ResetObject();
            }
        }
    }

    public void ResetGroup0()
    {
        ResetGroup(0);
    }

    public void ResetGroup1()
    {
        ResetGroup(1);
    }

    public void ResetGroup2()
    {
        ResetGroup(2);
    }

    public void ResetGroup3()
    {
        ResetGroup(3);
    }

    public void ResetGroup4()
    {
        ResetGroup(4);
    }

    public void ResetGroup5()
    {
        ResetGroup(5);
    }

    public void ResetGroup(int groupNumber)
    {
        if (resettableObjects == null)
        {
            return;
        }
    }
}

```


VRChat Paper Tablet UI Product Handleiding - v1.6 - StefanieInVR

```
for (int i = 0; i < resettableObjects.Length; i++)
{
    ResettableObject resetObject = resettableObjects[i];

    if (resetObject != null && resetObject.resetGroup == groupNumber)
    {
        resetObject.ResetObject();
    }
}
}
```

Appendix 16 - ResettableObject.cs

```

using UdonSharp;
using UnityEngine;
using VRC.SDKBase;
using VRC.SDK3.Components;

namespace StefanieInVR.PaperTablet
{
    public class ResettableObject : UdonSharpBehaviour
    {
        [Header("Basic Setup")]
        [Tooltip("The empty GameObject that marks where this object should return to.")]
        public Transform homeSpot;

        [Tooltip("Group number. Buttons reset objects by group number.")]
        public int resetGroup = 0;

        [Tooltip("OFF = local reset only. ON = reset should be visible for everyone. Use this with VRC Object Sync.")]
        public bool resetForEveryone = false;

        [Header("What Should Reset?")]
        [Tooltip("Reset the world position to the Home Spot.")]
        public bool resetPosition = true;

        [Tooltip("Reset the world rotation to the Home Spot.")]
        public bool resetRotation = true;

        [Tooltip("Reset the local scale to the Home Spot local scale. Usually leave this OFF.")]
        public bool resetScale = false;

        [Header("Pickup / Sync Options")]
        [Tooltip("If this object is a VRC Pickup, drop it before resetting.")]
        public bool dropPickupBeforeReset = true;

        [Tooltip("Optional. Fill this if VRC Pickup is not on the same GameObject as this script.")]
        public VRC_Pickup pickupObject;

        [Tooltip("Optional. Fill this if VRC Object Sync is not on the same GameObject as this script.")]
        public VRCObjectSync objectSync;

        private void Start()
        {
            AutoFindMissingReferences();
        }

        private void AutoFindMissingReferences()
        {
            if (pickupObject == null)
            {
                pickupObject = (VRC_Pickup)GetComponent(typeof(VRC_Pickup));
            }

            if (objectSync == null)
            {
                objectSync = (VRCObjectSync)GetComponent(typeof(VRCObjectSync));
            }
        }

        public void ResetObject()
        {
            if (homeSpot == null)
            {
                Debug.LogWarning("[ResettableObject] No Home Spot assigned on: " + gameObject.name);
                return;
            }

            AutoFindMissingReferences();
        }
    }
}

```

```

    if (resetForEveryone)
    {
        ResetForEveryone();
    }
    else
    {
        ResetLocalOnly();
    }
}

private void ResetLocalOnly()
{
    DropPickupIfNeeded();
    MoveToHomeSpot();
}

private void ResetForEveryone()
{
    if (Networking.LocalPlayer != null)
    {
        Networking.SetOwner(Networking.LocalPlayer, gameObject);

        if (pickupObject != null)
        {
            Networking.SetOwner(Networking.LocalPlayer, pickupObject.gameObject);
        }

        if (objectSync != null)
        {
            Networking.SetOwner(Networking.LocalPlayer, objectSync.gameObject);
            objectSync.FlagDiscontinuity();
        }
    }

    DropPickupIfNeeded();
    MoveToHomeSpot();

    if (objectSync != null)
    {
        objectSync.FlagDiscontinuity();
    }
}

private void DropPickupIfNeeded()
{
    if (!dropPickupBeforeReset)
    {
        return;
    }

    if (pickupObject == null)
    {
        return;
    }

    pickupObject.Drop();
}

private void MoveToHomeSpot()
{
    if (resetPosition)
    {
        transform.position = homeSpot.position;
    }

    if (resetRotation)
    {

```

VRChat Paper Tablet UI Product Handleiding - v1.6 - StefanieInVR

```
    transform.rotation = homeSpot.rotation;
}

if (resetScale)
{
    transform.localScale = homeSpot.localScale;
}
}
}
```

Appendix 17 - ScreenReadabilityInput.cs

```

using UdonSharp;
using UnityEngine;
using UnityEngine.UI;

namespace StefanieInVR
{
    [UdonBehaviourSyncMode(BehaviourSyncMode.None)]
    public class ScreenReadabilityInput : UdonSharpBehaviour
    {
        [Header("Manager")]
        public ScreenReadabilityManager manager;

        [Header("Input Type")]
        [Tooltip("Use this object as a slider input.")]
        public bool isSlider = true;

        [Tooltip("Use this object as a reset button input.")]
        public bool isResetButton = false;

        [Header("Control")]
        [Tooltip("0 = Brightness, 1 = Contrast")]
        public int controllIndex = 0;

        [Tooltip("If true, this input controls the global synced value. If false, it controls only the local player value.")]
        public bool global = false;

        [Header("UI References")]
        [Tooltip("Optional. If empty, the script tries to find a Slider on this GameObject.")]
        public Slider slider;

        [Header("Safety")]
        [Tooltip("Prevents RefreshFromManager from triggering slider logic again.")]
        public bool ignoreRefreshCallbacks = true;

        private bool isRefreshing;

        private void Start()
        {
            if (slider == null)
            {
                slider = (Slider)GetComponent(typeof(Slider));
            }

            RefreshFromManager();
        }

        public void OnSliderChanged()
        {
            if (!isSlider)
            {
                return;
            }

            if (isRefreshing && ignoreRefreshCallbacks)
            {
                return;
            }

            if (manager == null || slider == null)
            {
                return;
            }

            float value01 = slider.value;

            if (controllIndex == 0)

```

```

{
    if (global)
    {
        manager.SetGlobalBrightness01(value01);
    }
    else
    {
        manager.SetLocalBrightness01(value01);
    }

    return;
}

if (controlIndex == 1)
{
    if (global)
    {
        manager.SetGlobalContrast01(value01);
    }
    else
    {
        manager.SetLocalContrast01(value01);
    }

    return;
}
}

public void OnResetPressed()
{
    if (!isResetButton)
    {
        return;
    }

    if (manager == null)
    {
        return;
    }

    if (global)
    {
        manager.ResetGlobal();
    }
    else
    {
        manager.ResetLocal();
    }
}

public void RefreshFromManager()
{
    if (manager == null)
    {
        return;
    }

    if (!isSlider)
    {
        return;
    }

    if (slider == null)
    {
        slider = (Slider)GetComponent(typeof(Slider));
    }

    if (slider == null)

```

```
{
    return;
}

isRefreshing = true;

if (controlIndex == 0)
{
    slider.value = manager.GetBrightness01(global);
}
else if (controlIndex == 1)
{
    slider.value = manager.GetContrast01(global);
}

isRefreshing = false;
}
}
}
```

Appendix 18 - ScreenReadabilityManager.cs

```

using UdonSharp;
using UnityEngine;
using VRC.SDKBase;

namespace StefanieInVR
{
    [UdonBehaviourSyncMode(BehaviourSyncMode.Manual)]
    public class ScreenReadabilityManager : UdonSharpBehaviour
    {
        [Header("Screen Materials")]
        [Tooltip("All materials that should receive brightness/contrast changes. Usually your shared video screen material.")]
        public Material[] targetMaterials;

        [Header("Shader Property Names")]
        [Tooltip("Brightness float property on the screen shader.")]
        public string brightnessProperty = "_Brightness";

        [Tooltip("Contrast float property on the screen shader.")]
        public string contrastProperty = "_Contrast";

        [Header("Default Values")]
        [Range(0f, 2f)]
        public float defaultBrightness = 1f;

        [Range(0f, 2f)]
        public float defaultContrast = 1f;

        [Header("Limits")]
        [Range(0f, 2f)]
        public float minBrightness = 0f;

        [Range(0f, 2f)]
        public float maxBrightness = 2f;

        [Range(0f, 2f)]
        public float minContrast = 0f;

        [Range(0f, 2f)]
        public float maxContrast = 2f;

        [Header("Global Synced Values")]
        [UdonSynced]
        private float globalBrightness = 1f;

        [UdonSynced]
        private float globalContrast = 1f;

        [Header("Linked UI Inputs")]
        [Tooltip("All ScreenReadabilityInput scripts that should visually update when values change.")]
        public ScreenReadabilityInput[] linkedInputs;

        [Header("Network Sync")]
        [Tooltip("Delay before sending global slider changes over the network. Prevents network spam while dragging.")]
        public float syncDelay = 0.35f;

        [Header("Debug")]
        public bool debugLogs = false;

        private float localBrightness = 1f;
        private float localContrast = 1f;

        private bool waitingForOwnership;
        private int ownershipRetryCount;
        private const int MaxOwnershipRetries = 10;

        private bool delayedSyncScheduled;
    }
}

```


VRChat Paper Tablet UI Product Handleiding - v1.6 - StefanieInVR

```
private float lastGlobalChangeTime;

private void Start()
{
    globalBrightness = ClampBrightness(globalBrightness);
    globalContrast = ClampContrast(globalContrast);

    localBrightness = globalBrightness;
    localContrast = globalContrast;

    ApplyValues();
    RefreshLinkedInputs();

    if (debugLogs)
    {
        Debug.Log("[ScreenReadabilityManager] Started.");
    }
}

public override void OnDeserialization()
{
    globalBrightness = ClampBrightness(globalBrightness);
    globalContrast = ClampContrast(globalContrast);

    // A global change overrules local values on every client.
    localBrightness = globalBrightness;
    localContrast = globalContrast;

    ApplyValues();
    RefreshLinkedInputs();

    if (debugLogs)
    {
        Debug.Log("[ScreenReadabilityManager] Received global values from network.");
    }
}

public void SetGlobalBrightness01(float normalizedValue)
{
    SetGlobalBrightness(Convert01ToBrightness(normalizedValue));
}

public void SetGlobalContrast01(float normalizedValue)
{
    SetGlobalContrast(Convert01ToContrast(normalizedValue));
}

public void SetLocalBrightness01(float normalizedValue)
{
    SetLocalBrightness(Convert01ToBrightness(normalizedValue));
}

public void SetLocalContrast01(float normalizedValue)
{
    SetLocalContrast(Convert01ToContrast(normalizedValue));
}

public void SetGlobalBrightness(float value)
{
    globalBrightness = ClampBrightness(value);

    // Global overrules local immediately on this client too.
    localBrightness = globalBrightness;

    ApplyValues();
    RefreshLinkedInputs();
    ScheduleDelayedGlobalSync();
}
```

VRChat Paper Tablet UI Product Handleiding - v1.6 - StefanieInVR

```
if (debugLogs)
{
    Debug.Log("[ScreenReadabilityManager] Global brightness changed locally, delayed sync scheduled.");
}
}

public void SetGlobalContrast(float value)
{
    globalContrast = ClampContrast(value);

    // Global overrules local immediately on this client too.
    localContrast = globalContrast;

    ApplyValues();
    RefreshLinkedInputs();
    ScheduleDelayedGlobalSync();

    if (debugLogs)
    {
        Debug.Log("[ScreenReadabilityManager] Global contrast changed locally, delayed sync scheduled.");
    }
}

public void SetLocalBrightness(float value)
{
    localBrightness = ClampBrightness(value);

    ApplyValues();
    RefreshLinkedInputs();

    if (debugLogs)
    {
        Debug.Log("[ScreenReadabilityManager] Local brightness changed.");
    }
}

public void SetLocalContrast(float value)
{
    localContrast = ClampContrast(value);

    ApplyValues();
    RefreshLinkedInputs();

    if (debugLogs)
    {
        Debug.Log("[ScreenReadabilityManager] Local contrast changed.");
    }
}

public void ResetGlobal()
{
    globalBrightness = ClampBrightness(defaultBrightness);
    globalContrast = ClampContrast(defaultContrast);

    // Global reset overrules local values.
    localBrightness = globalBrightness;
    localContrast = globalContrast;

    ApplyValues();
    RefreshLinkedInputs();

    // Reset is a button press, so sync immediately.
    RequestSafeSerialization();

    if (debugLogs)
    {
        Debug.Log("[ScreenReadabilityManager] Global reset.");
    }
}
```

```

}

public void ResetLocal()
{
    localBrightness = ClampBrightness(defaultBrightness);
    localContrast = ClampContrast(defaultContrast);

    ApplyValues();
    RefreshLinkedInputs();

    if (debugLogs)
    {
        Debug.Log("[ScreenReadabilityManager] Local reset.");
    }
}

public float GetGlobalBrightness()
{
    return globalBrightness;
}

public float GetGlobalContrast()
{
    return globalContrast;
}

public float GetLocalBrightness()
{
    return localBrightness;
}

public float GetLocalContrast()
{
    return localContrast;
}

public float GetBrightness01(bool useGlobal)
{
    float value = useGlobal ? globalBrightness : localBrightness;
    return ConvertBrightnessTo01(value);
}

public float GetContrast01(bool useGlobal)
{
    float value = useGlobal ? globalContrast : localContrast;
    return ConvertContrastTo01(value);
}

public void RefreshLinkedInputs()
{
    if (linkedInputs == null)
    {
        return;
    }

    for (int i = 0; i < linkedInputs.Length; i++)
    {
        ScreenReadabilityInput input = linkedInputs[i];

        if (input == null)
        {
            continue;
        }

        input.RefreshFromManager();
    }
}

```

```

private void ApplyValues()
{
    if (targetMaterials == null)
    {
        return;
    }

    for (int i = 0; i < targetMaterials.Length; i++)
    {
        Material mat = targetMaterials[i];

        if (mat == null)
        {
            continue;
        }

        if (!string.IsNullOrEmpty(brightnessProperty) && mat.HasProperty(brightnessProperty))
        {
            mat.SetFloat(brightnessProperty, localBrightness);
        }

        if (!string.IsNullOrEmpty(contrastProperty) && mat.HasProperty(contrastProperty))
        {
            mat.SetFloat(contrastProperty, localContrast);
        }
    }
}

private float ClampBrightness(float value)
{
    return Mathf.Clamp(value, minBrightness, maxBrightness);
}

private float ClampContrast(float value)
{
    return Mathf.Clamp(value, minContrast, maxContrast);
}

private float Convert01ToBrightness(float normalizedValue)
{
    return Mathf.Lerp(minBrightness, maxBrightness, Mathf.Clamp01(normalizedValue));
}

private float Convert01ToContrast(float normalizedValue)
{
    return Mathf.Lerp(minContrast, maxContrast, Mathf.Clamp01(normalizedValue));
}

private float ConvertBrightnessTo01(float value)
{
    if (Mathf.Approximately(maxBrightness, minBrightness))
    {
        return 0f;
    }

    return Mathf.InverseLerp(minBrightness, maxBrightness, ClampBrightness(value));
}

private float ConvertContrastTo01(float value)
{
    if (Mathf.Approximately(maxContrast, minContrast))
    {
        return 0f;
    }

    return Mathf.InverseLerp(minContrast, maxContrast, ClampContrast(value));
}

```

VRChat Paper Tablet UI Product Handleiding - v1.6 - StefanieInVR

```
private void ScheduleDelayedGlobalSync()
{
    lastGlobalChangeTime = Time.time;

    if (delayedSyncScheduled)
    {
        return;
    }

    delayedSyncScheduled = true;
    SendCustomEventDelayedSeconds(nameof(DelayedGlobalSyncCheck), syncDelay);
}

public void DelayedGlobalSyncCheck()
{
    float timeSinceLastChange = Time.time - lastGlobalChangeTime;

    if (timeSinceLastChange < syncDelay)
    {
        SendCustomEventDelayedSeconds(nameof(DelayedGlobalSyncCheck), syncDelay);
        return;
    }

    delayedSyncScheduled = false;
    RequestSafeSerialization();

    if (debugLogs)
    {
        Debug.Log("[ScreenReadabilityManager] Delayed global sync requested.");
    }
}

private void RequestSafeSerialization()
{
    if (Networking.LocalPlayer == null)
    {
        RequestSerialization();
        return;
    }

    if (Networking.IsOwner(gameObject))
    {
        RequestSerialization();
        return;
    }

    waitingForOwnership = true;
    ownershipRetryCount = 0;

    Networking.SetOwner(Networking.LocalPlayer, gameObject);
    SendCustomEventDelayedFrames(nameof(DelayedRequestSerialization), 1);
}

public void DelayedRequestSerialization()
{
    if (!waitingForOwnership)
    {
        return;
    }

    if (Networking.LocalPlayer == null)
    {
        waitingForOwnership = false;
        RequestSerialization();
        return;
    }

    if (Networking.IsOwner(gameObject))
```

VRChat Paper Tablet UI Product Handleiding - v1.6 - StefanieInVR

```
{
    waitingForOwnership = false;
    RequestSerialization();

    if (debugLogs)
    {
        Debug.Log("[ScreenReadabilityManager] Serialization requested after ownership transfer.");
    }

    return;
}

ownershipRetryCount++;

if (ownershipRetryCount >= MaxOwnershipRetries)
{
    waitingForOwnership = false;

    if (debugLogs)
    {
        Debug.LogWarning("[ScreenReadabilityManager] Could not get ownership in time.");
    }

    return;
}

SendCustomEventDelayedFrames(nameof(DelayedRequestSerialization), 1);
}
}
}
```

Appendix 19 - TXLScreenAutoVisibility.cs

```

using UdonSharp;
using UnityEngine;
using VRC.SDKBase;

namespace StefanieInVR.PaperTablet { }

[UdonBehaviourSyncMode(BehaviourSyncMode.None)]
public class TXLScreenAutoVisibility : UdonSharpBehaviour
{
    [Header("Required")]
    [Tooltip("Your TXL SyncPlayer component.")]
    public Texel.SyncPlayer txlPlayer;

    [Tooltip("The TXL screen plane object with MeshRenderer.")]
    public GameObject txlScreenObject;

    [Tooltip("The projector screen cloth object with SkinnedMeshRenderer and blendshape.")]
    public GameObject projectorScreenObject;

    [Header("Projector Gate")]
    [Tooltip("Same blendshape index as the projector toggle script.")]
    public int blendShapeIndex = 0;

    [Tooltip("The projector screen counts as open above this blendshape value.")]
    [Range(0f, 100f)]
    public float openThreshold = 99.5f;

    [Header("TXL Gate")]
    [Tooltip("Show the TXL screen while video is loading, but only when the projector screen is open.")]
    public bool showWhenLoading = true;

    [Tooltip("Keep the TXL screen visible while paused.")]
    public bool showWhenPaused = true;

    [Header("Collider Control")]
    [Tooltip("Enable/disable colliders on the TXL screen object together with the renderer.")]
    public bool controlColliders = true;

    [Tooltip("Also include child colliders.")]
    public bool includeChildColliders = false;

    [Header("Performance")]
    [Tooltip("How often visibility is checked. 0.25 is usually light and fast enough.")]
    [Range(0.05f, 1f)]
    public float pollInterval = 0.25f;

    private Renderer screenRenderer;
    private SkinnedMeshRenderer projectorRenderer;
    private Collider[] screenColliders;

    private float nextPollTime;
    private bool lastVisible;

    private void Start()
    {
        CacheComponents();
        ApplyVisibility(true);
    }

    private void Update()
    {
        if (Time.time < nextPollTime)
        {
            return;
        }
    }
}

```

VRChat Paper Tablet UI Product Handleiding - v1.6 - StefanieInVR

```
nextPollTime = Time.time + pollInterval;
ApplyVisibility(false);
}

private void CacheComponents()
{
    if (txlScreenObject != null)
    {
        screenRenderer = txlScreenObject.GetComponent<Renderer>();

        if (controlColliders)
        {
            if (includeChildColliders)
            {
                screenColliders = txlScreenObject.GetComponentsInChildren<Collider>(true);
            }
            else
            {
                screenColliders = txlScreenObject.GetComponents<Collider>();
            }
        }
    }

    if (projectorScreenObject != null)
    {
        projectorRenderer = projectorScreenObject.GetComponent<SkinnedMeshRenderer>();
    }
}

private void ApplyVisibility(bool force)
{
    if (!Utilities.IsValid(txlPlayer))
    {
        return;
    }

    if (screenRenderer == null || projectorRenderer == null)
    {
        return;
    }

    float weight = projectorRenderer.GetBlendShapeWeight(blendShapeIndex);
    bool projectorOpen = weight >= openThreshold;

    int state = txlPlayer.playerState;
    bool txlShouldBeVisible = false;

    if (state == Texel.TXLVideoPlayer.VIDEO_STATE_LOADING)
    {
        txlShouldBeVisible = showWhenLoading;
    }
    else if (state == Texel.TXLVideoPlayer.VIDEO_STATE_PLAYING)
    {
        txlShouldBeVisible = txlPlayer.paused ? showWhenPaused : true;
    }
    else
    {
        txlShouldBeVisible = false;
    }

    bool visible = projectorOpen && txlShouldBeVisible;

    if (!force && visible == lastVisible)
    {
        return;
    }

    screenRenderer.enabled = visible;
```


VRChat Paper Tablet UI Product Handleiding - v1.6 - StefanieInVR

```
if (controlColliders && screenColliders != null)
{
    for (int i = 0; i < screenColliders.Length; i++)
    {
        if (screenColliders[i] != null)
        {
            screenColliders[i].enabled = visible;
        }
    }
}

lastVisible = visible;
}
```

Appendix 20 - VipAccessManager.cs

```

using UdonSharp;
using UnityEngine;
using UnityEngine.UI;
using TMPro;
using VRC.SDKBase;
using VRC.SDK3.StringLoading;
using VRC.Udon.Common.Interfaces;

[UdonBehaviourSyncMode(BehaviourSyncMode.Manual)]
public class VipAccessManager : UdonSharpBehaviour
{
    [Header("1. VIP List")]
    [Tooltip("URL naar een simpele tekstlijst met VRChat display names. Een naam per regel.")]
    public VRCUrl vipListUrl;

    [Header("2. Tablet / Tab")]
    [Tooltip("Je PaperTabletTabManager.")]
    public PaperTabletTabManager tabletTabManager;

    [Tooltip("Welke tabIndex is de VIP-tab?")]
    public int vipTabIndex = 6;

    [Header("3. VIP Tab Button")]
    [Tooltip("De zichtbare VIP-tab button.")]
    public Button vipTabButton;

    [Header("4. VIP Panel Content")]
    [Tooltip("Echte VIP-inhoud. Alleen zichtbaar voor spelers op de VIP-lijst wanneer de VIP-tab open is.")]
    public GameObject vipContentRoot;

    [Tooltip("Niet meer hoofdlogica. Mag leeg blijven als je nu alleen TabletLockedRoot gebruikt.")]
    public GameObject nonVipDummyRoot;

    [Header("5. Tablet Locked Overlay")]
    [Tooltip("Algemeen locked-scherm dat niet-VIPs zien wanneer Tablet Lock aan staat, of wanneer zij de VIP-tab openen.")]
    public GameObject tabletLockedRoot;

    [Header("6. Restricted Objects On VIP Tab")]
    [Tooltip("Extra objecten die zeker uit moeten voor niet-VIPs wanneer TabletLockedRoot wordt getoond. Bijvoorbeeld losse PlayerControls als die niet goed via TabState worden uitgezet.")]
    public GameObject[] restrictedVipTabObjects;

    [Header("7. Refresh")]
    [Tooltip("Knop onder VipContentRoot waarmee VIPs/admins de lijst opnieuw laten laden voor iedereen.")]
    public Button refreshVipListButton;

    [Tooltip("TextMeshPro tekstveld om voor VIPs te tonen of refresh gelukt is.")]
    public TextMeshProUGUI refreshStatusText;

    [Header("8. Optional Local VIP Objects")]
    [Tooltip("Deze objecten gaan lokaal AAN als de speler VIP is, ongeacht welke tab open is.")]
    public GameObject[] enableWhenVip;

    [Tooltip("Deze objecten gaan lokaal UIT als de speler VIP is, en AAN als de speler geen VIP is.")]
    public GameObject[] disableWhenVip;

    [Header("9. Tablet Lock")]
    [Tooltip("Gesyncte tablet lock status. Als deze aan staat, mogen niet-VIPs geen tabs bedienen en zien ze alleen TabletLockedRoot.")]
    [UdonSynced] public bool tabletLockEnabled = false;

    [Tooltip("Knop waarmee VIPs/admins de tablet lock aan/uit zetten.")]
    public Button tabletLockButton;

    [Tooltip("Los TextMeshPro veld dat Tablet Locked / Tablet Unlocked toont.")]
    public TextMeshProUGUI tabletLockStatusText;

```

VRChat Paper Tablet UI Product Handleiding - v1.6 - StefanieInVR

```
[Tooltip("Image component van de tablet lock knop waarvan de sprite gewisseld moet worden.")]
public Image tabletLockButtonImage;

[Tooltip("Sprite wanneer de tablet locked is.")]
public Sprite tabletLockOnSprite;

[Tooltip("Sprite wanneer de tablet unlocked is.")]
public Sprite tabletLockOffSprite;

[Header("10. Stage Blocker")]
[Tooltip("Gesyncte status. Als deze aan staat, krijgen niet-VIPs lokaal de stage blockers aan. VIPs blijven erdoorheen kunnen.")]
[UdonSynced] public bool stageBlockerEnabled = true;

[Tooltip("Deze objecten gaan lokaal AAN voor niet-VIPs wanneer Stage Blocker aan staat. Voor VIPs blijven ze UIT.")]
public GameObject[] stageBlockerObjects;

[Tooltip("Optionele TextMeshPro tekst om de stage blocker status te tonen.")]
public TextMeshProUGUI stageBlockerStatusText;

[Header("10B. Stage Blocker Button Visual")]
[Tooltip("Image component van de stage blocker knop waarvan de sprite gewisseld moet worden.")]
public Image stageBlockerButtonImage;

[Tooltip("Sprite wanneer de stage blocker AAN staat.")]
public Sprite stageBlockerOnSprite;

[Tooltip("Sprite wanneer de stage blocker UIT staat.")]
public Sprite stageBlockerOffSprite;

[Header("11. Optional Debug")]
[Tooltip("Alleen handig tijdens testen. Zet uit voor release.")]
public bool debugLogs = false;

[UdonSynced] private int refreshVersion = 0;

private bool isVip = false;
private bool hasLoadedList = false;

private int localKnownRefreshVersion = -1;
private bool localKnownTabletLockEnabled = false;
private bool localKnownStageBlockerEnabled = true;

private string lastLoadedListText = "";
private bool hasPreviousListText = false;
private bool currentLoadWasRefresh = false;

private bool pendingSerialization = false;
private int serializationTries = 0;

private void Start()
{
    localKnownTabletLockEnabled = tabletLockEnabled;
    localKnownStageBlockerEnabled = stageBlockerEnabled;

    ApplyAccessBase(false);
    ApplyTabletLockVisual();
    ApplyTabletLockedOverlay();
    ApplyStageBlockerAccess();
    UpdateStageBlockerStatusAndVisual();

    SetStatus("Loading VIP list...");

    localKnownRefreshVersion = refreshVersion;

    LoadVipList(false);
}

public override void OnDeserialization()
```

VRChat Paper Tablet UI Product Handleiding - v1.6 - StefanieInVR

```
{
    bool tabletLockChanged = localKnownTabletLockEnabled != tabletLockEnabled;
    bool stageBlockerChanged = localKnownStageBlockerEnabled != stageBlockerEnabled;

    localKnownTabletLockEnabled = tabletLockEnabled;
    localKnownStageBlockerEnabled = stageBlockerEnabled;

    ApplyTabletLockVisual();
    ApplyStageBlockerAccess();
    UpdateStageBlockerStatusAndVisual();

    if (tabletLockChanged || stageBlockerChanged)
    {
        if (tabletTabManager != null)
        {
            tabletTabManager.RefreshAll();
        }
        else
        {
            RefreshCurrentTabFilter();
        }
    }

    if (localKnownRefreshVersion == refreshVersion)
    {
        return;
    }

    localKnownRefreshVersion = refreshVersion;

    if (debugLogs)
    {
        Debug.Log("[VipAccessManager] Refresh signal received. Version: " + refreshVersion);
    }

    LoadVipList(true);
}

public override void OnStringLoadSuccess(IVRCStringDownload result)
{
    hasLoadedList = true;

    string newListText = result.Result;
    string localName = "";

    if (Networking.LocalPlayer != null)
    {
        localName = Networking.LocalPlayer.displayName;
    }

    bool previousVipState = isVip;

    string addedNames = "";
    string removedNames = "";
    bool listChanged = false;

    if (hasPreviousListText)
    {
        listChanged = lastLoadedListText != newListText;

        if (listChanged)
        {
            addedNames = GetAddedNames(lastLoadedListText, newListText);
            removedNames = GetRemovedNames(lastLoadedListText, newListText);
        }
    }

    lastLoadedListText = newListText;
}
```

VRChat Paper Tablet UI Product Handleiding - v1.6 - StefanieInVR

```
hasPreviousListText = true;

isVip = IsNameInList(localName, newListText);

ApplyAccessBase(isVip);

if (tabletTabManager != null)
{
    tabletTabManager.RefreshAll();
}
else
{
    RefreshCurrentTabFilter();
}

ApplyTabletLockVisual();
ApplyTabletLockedOverlay();
ApplyStageBlockerAccess();
UpdateStageBlockerStatusAndVisual();

if (debugLogs)
{
    Debug.Log("[VipAccessManager] Local player: " + localName + " | Is VIP: " + isVip + " | List changed: " + listChanged);
}

if (currentLoadWasRefresh)
{
    if (listChanged)
    {
        string message = "VIP list updated.";

        if (!string.IsNullOrEmpty(addedNames))
        {
            message += "\nAdded: " + LimitText(addedNames, 160);
        }

        if (!string.IsNullOrEmpty(removedNames))
        {
            message += "\nRemoved: " + LimitText(removedNames, 160);
        }

        if (string.IsNullOrEmpty(addedNames) && string.IsNullOrEmpty(removedNames))
        {
            message += "\nOnly comments/spacing changed.";
        }

        if (previousVipState != isVip)
        {
            message += isVip ? "\nYou are now VIP." : "\nYou are no longer VIP.";
        }
        else
        {
            message += isVip ? "\nYour access: VIP." : "\nYour access: not VIP.";
        }

        SetStatus(message);
    }
    else
    {
        SetStatus(isVip
            ? "VIP list reloaded.\nNo name changes found.\nYour access: VIP."
            : "VIP list reloaded.\nNo name changes found.\nYour access: not VIP.");
    }
}
else
{
    SetStatus(isVip
        ? "VIP access loaded.\nYour access: VIP."
        : "VIP access loaded.\nYour access: not VIP.");
}
```

VRChat Paper Tablet UI Product Handleiding - v1.6 - StefanieInVR

```
        : "VIP access loaded.\nYour access: not VIP.");
    }

    currentLoadWasRefresh = false;
}

public override void OnStringLoadError(IVRCStringDownload result)
{
    hasLoadedList = false;
    isVip = false;
    currentLoadWasRefresh = false;

    if (debugLogs)
    {
        Debug.Log("[VipAccessManager] Could not load VIP list. Error: " + result.ErrorCode + " - " + result.Error);
    }

    ApplyAccessBase(false);

    if (tabletTabManager != null)
    {
        tabletTabManager.RefreshAll();
    }
    else
    {
        RefreshCurrentTabFilter();
    }

    ApplyTabletLockVisual();
    ApplyTabletLockedOverlay();
    ApplyStageBlockerAccess();
    UpdateStageBlockerStatusAndVisual();

    SetStatus("Could not load VIP list.");
}

public void RequestVipListRefreshForEveryone()
{
    if (!isVip)
    {
        SetStatus("Only VIPs can refresh the VIP list for everyone.");
        return;
    }

    TakeOwnershipIfNeeded();
    if (!Networking.IsOwner(gameObject)) return;

    refreshVersion++;
    localKnownRefreshVersion = refreshVersion;

    RequestSerializationSafe();

    if (debugLogs)
    {
        Debug.Log("[VipAccessManager] Refresh requested for everyone. Version: " + refreshVersion);
    }

    SetStatus("Refreshing VIP list for everyone...");
    LoadVipList(true);
}

public void RefreshVipListLocalOnly()
{
    SetStatus("Refreshing VIP list locally...");
    LoadVipList(true);
}

public void ToggleTabletLockForEveryone()
```

```

{
    if (!isVip)
    {
        SetStatus("Only VIPs can lock or unlock the tablet.");
        return;
    }

    TakeOwnershipIfNeeded();
    if (!Networking.IsOwner(gameObject)) return;

    tabletLockEnabled = !tabletLockEnabled;
    localKnownTabletLockEnabled = tabletLockEnabled;

    RequestSerializationSafe();

    if (tabletTabManager != null)
    {
        tabletTabManager.RefreshAll();
    }
    else
    {
        RefreshCurrentTabFilter();
    }

    ApplyTabletLockVisual();
    ApplyTabletLockedOverlay();

    if (debugLogs)
    {
        Debug.Log("[VipAccessManager] Tablet lock toggled. Locked: " + tabletLockEnabled);
    }
}

public void ToggleStageBlockerForEveryone()
{
    if (!isVip)
    {
        SetStatus("Only VIPs can change the stage blocker.");
        return;
    }

    TakeOwnershipIfNeeded();
    if (!Networking.IsOwner(gameObject)) return;

    stageBlockerEnabled = !stageBlockerEnabled;
    localKnownStageBlockerEnabled = stageBlockerEnabled;

    RequestSerializationSafe();

    ApplyStageBlockerAccess();
    UpdateStageBlockerStatusAndVisual();

    if (debugLogs)
    {
        Debug.Log("[VipAccessManager] Stage blocker toggled. Enabled: " + stageBlockerEnabled);
    }
}

private void TakeOwnershipIfNeeded()
{
    if (Networking.IsOwner(gameObject)) return;
    if (Networking.LocalPlayer == null) return;

    Networking.SetOwner(Networking.LocalPlayer, gameObject);
}

private void RequestSerializationSafe()
{

```

VRChat Paper Tablet UI Product Handleiding - v1.6 - StefanieInVR

```
    pendingSerialization = true;
    serializationTries = 0;
    SendCustomEventDelayedFrames(nameof(DelayedRequestSerialization), 1);
}

public void DelayedRequestSerialization()
{
    if (!pendingSerialization) return;

    if (!Networking.IsOwner(gameObject))
    {
        serializationTries++;

        if (serializationTries < 10)
        {
            SendCustomEventDelayedFrames(nameof(DelayedRequestSerialization), 1);
        }

        return;
    }

    pendingSerialization = false;
    RequestSerialization();
}

public bool IsLocalPlayerVip()
{
    return isVip;
}

public bool HasLoadedList()
{
    return hasLoadedList;
}

public bool IsTabletLocked()
{
    return tabletLockEnabled;
}

public bool IsStageBlockerEnabled()
{
    return stageBlockerEnabled;
}

public bool ShouldHideTabletContentForLocalPlayer(int currentTabIndex)
{
    if (isVip)
    {
        return false;
    }

    if (tabletLockEnabled)
    {
        return true;
    }

    if (currentTabIndex == vipTabIndex)
    {
        return true;
    }

    return false;
}

public bool CanLocalPlayerSelectTab(int requestedTabIndex, int currentTabIndex)
{
    if (isVip)
```



```

{
    return true;
}

if (tabletLockEnabled)
{
    return false;
}

return true;
}

public void ApplyAccessForCurrentTab(int currentTabIndex)
{
    bool showOnlyLockedRoot = ShouldHideTabletContentForLocalPlayer(currentTabIndex);

    if (vipTabButton != null)
    {
        vipTabButton.interactable = true;
    }

    if (tabletLockButton != null)
    {
        tabletLockButton.interactable = isVip;
    }

    if (refreshVipListButton != null)
    {
        refreshVipListButton.interactable = isVip;
    }

    SetObjectsActive(enableWhenVip, isVip);
    SetObjectsActive(disableWhenVip, !isVip);

    ApplyTabletLockVisual();
    ApplyTabletLockedOverlay();
    ApplyStageBlockerAccess();
    UpdateStageBlockerStatusAndVisual();

    if (showOnlyLockedRoot)
    {
        SetActiveIfNeeded(vipContentRoot, false);
        SetActiveIfNeeded(nonVipDummyRoot, false);
        SetObjectsActive(restrictedVipTabObjects, false);
        return;
    }

    if (currentTabIndex != vipTabIndex)
    {
        SetActiveIfNeeded(vipContentRoot, false);
        SetActiveIfNeeded(nonVipDummyRoot, false);
        return;
    }

    SetActiveIfNeeded(vipContentRoot, isVip);
    SetActiveIfNeeded(nonVipDummyRoot, !isVip);
    SetObjectsActive(restrictedVipTabObjects, isVip);
}

private void RefreshCurrentTabFilter()
{
    if (tabletTabManager != null)
    {
        ApplyAccessForCurrentTab(tabletTabManager.GetSelectedTabIndex());
    }
    else
    {
        ApplyAccessForCurrentTab(-1);
    }
}

```

```

    }
}

private void LoadVipList(bool isRefresh)
{
    currentLoadWasRefresh = isRefresh;

    if (vipListUrl == null || string.IsNullOrEmpty(vipListUrl.ToString()))
    {
        ApplyAccessBase(false);

        if (tabletTabManager != null)
        {
            tabletTabManager.RefreshAll();
        }
        else
        {
            RefreshCurrentTabFilter();
        }

        ApplyTabletLockVisual();
        ApplyTabletLockedOverlay();
        ApplyStageBlockerAccess();
        UpdateStageBlockerStatusAndVisual();

        SetStatus("No VIP list URL set.");
        return;
    }

    VRCStringDownloader.LoadUrl(vipListUrl, (IUDonEventReceiver)this);
}

private void ApplyAccessBase(bool allowVipAccess)
{
    if (refreshVipListButton != null)
    {
        refreshVipListButton.interactable = allowVipAccess;
    }

    if (tabletLockButton != null)
    {
        tabletLockButton.interactable = allowVipAccess;
    }

    SetObjectsActive(enableWhenVip, allowVipAccess);
    SetObjectsActive(disableWhenVip, !allowVipAccess);
}

private void ApplyTabletLockVisual()
{
    if (tabletLockStatusText != null)
    {
        tabletLockStatusText.text = tabletLockEnabled ? "Tablet Locked" : "Tablet Unlocked";
    }

    if (tabletLockButtonImage != null)
    {
        if (tabletLockEnabled)
        {
            if (tabletLockOnSprite != null)
            {
                tabletLockButtonImage.sprite = tabletLockOnSprite;
            }
        }
        else
        {
            if (tabletLockOffSprite != null)
            {

```

VRChat Paper Tablet UI Product Handleiding - v1.6 - StefanieInVR

```
        tabletLockButtonImage.sprite = tabletLockOffSprite;
    }
}
}

private void ApplyTabletLockedOverlay()
{
    int currentTabIndex = -1;

    if (tabletTabManager != null)
    {
        currentTabIndex = tabletTabManager.GetSelectedTabIndex();
    }

    SetActiveIfNeeded(tabletLockedRoot, ShouldHideTabletContentForLocalPlayer(currentTabIndex));
}

private void ApplyStageBlockerAccess()
{
    bool shouldBlockThisPlayer = stageBlockerEnabled && !isVip;

    SetObjectsActive(stageBlockerObjects, shouldBlockThisPlayer);
}

private void UpdateStageBlockerStatusAndVisual()
{
    if (stageBlockerStatusText != null)
    {
        stageBlockerStatusText.text = stageBlockerEnabled
            ? "Stage blocker: ON for non-VIPs"
            : "Stage blocker: OFF";
    }

    if (stageBlockerButtonImage != null)
    {
        if (stageBlockerEnabled)
        {
            if (stageBlockerOnSprite != null)
            {
                stageBlockerButtonImage.sprite = stageBlockerOnSprite;
            }
        }
        else
        {
            if (stageBlockerOffSprite != null)
            {
                stageBlockerButtonImage.sprite = stageBlockerOffSprite;
            }
        }
    }
}

private bool IsNameInList(string playerName, string listText)
{
    if (string.IsNullOrEmpty(playerName)) return false;
    if (string.IsNullOrEmpty(listText)) return false;

    string cleanPlayerName = NormalizeName(playerName);

    string[] lines = listText.Split('\n');

    for (int i = 0; i < lines.Length; i++)
    {
        string line = NormalizeName(lines[i]);

        if (string.IsNullOrEmpty(line)) continue;
        if (line.StartsWith("#")) continue;
    }
}
```

```

    if (line == cleanPlayerName)
    {
        return true;
    }
}

return false;
}

private string GetAddedNames(string oldText, string newText)
{
    string result = "";

    string[] newLines = newText.Split("\n");

    for (int i = 0; i < newLines.Length; i++)
    {
        string newName = CleanListName(newLines[i]);

        if (string.IsNullOrEmpty(newName)) continue;

        if (!IsNameInList(newName, oldText))
        {
            result = AddNameToResult(result, newName);
        }
    }

    return result;
}

private string GetRemovedNames(string oldText, string newText)
{
    string result = "";

    string[] oldLines = oldText.Split("\n");

    for (int i = 0; i < oldLines.Length; i++)
    {
        string oldName = CleanListName(oldLines[i]);

        if (string.IsNullOrEmpty(oldName)) continue;

        if (!IsNameInList(oldName, newText))
        {
            result = AddNameToResult(result, oldName);
        }
    }

    return result;
}

private string CleanListName(string value)
{
    if (string.IsNullOrEmpty(value)) return "";

    string clean = value.Replace("\r", "").Trim();

    if (string.IsNullOrEmpty(clean)) return "";
    if (clean.StartsWith("#")) return "";

    return clean;
}

private string AddNameToResult(string currentResult, string nameToAdd)
{
    if (string.IsNullOrEmpty(currentResult))
    {

```

```
        return nameToAdd;
    }

    return currentResult + ", " + nameToAdd;
}

private string NormalizeName(string value)
{
    if (string.IsNullOrEmpty(value)) return "";

    return value.Replace("\r", "").Trim().ToLower();
}

private string LimitText(string value, int maxLength)
{
    if (string.IsNullOrEmpty(value)) return "";

    if (value.Length <= maxLength)
    {
        return value;
    }

    return value.Substring(0, maxLength) + "...";
}

private void SetStatus(string message)
{
    if (refreshStatusText != null)
    {
        refreshStatusText.text = message;
    }
}

private void SetObjectsActive(GameObject[] objects, bool active)
{
    if (objects == null) return;

    for (int i = 0; i < objects.Length; i++)
    {
        SetActiveIfNeeded(objects[i], active);
    }
}

private void SetActiveIfNeeded(GameObject target, bool active)
{
    if (target == null) return;

    if (target.activeSelf != active)
    {
        target.SetActive(active);
    }
}
}
```

Appendix 21 - NovaHierarchyQuickPanel.cs

```

#if UNITY_EDITOR
using System;
using System.Collections.Generic;
using System.Reflection;
using UnityEditor;
using UnityEngine;
using UnityEngine.SceneManagement;

public class NovaHierarchyQuickPanel : EditorWindow
{
    [Serializable]
    private class BookmarkData
    {
        public string label;
        public string globalId;
    }

    [Serializable]
    private class BookmarkList
    {
        public List<BookmarkData> bookmarks = new List<BookmarkData>();
    }

    private const string EditorPrefsKey = "NovaHierarchyQuickPanel_Bookmarks_v4";

    private BookmarkList data = new BookmarkList();

    private readonly Dictionary<string, UnityEngine.Object> objectCache =
        new Dictionary<string, UnityEngine.Object>();

    private Vector2 scrollPosition;
    private string searchText = "";

    private Type hierarchyWindowType;
    private MethodInfo setExpandedMethod;

    [MenuItem("Nova/Editor Tools/Hierarchy Quick Panel")]
    public static void OpenWindow()
    {
        NovaHierarchyQuickPanel window = GetWindow<NovaHierarchyQuickPanel>();
        window.titleContent = new GUIContent("Nova Hierarchy");
        window.minSize = new Vector2(360, 300);
        window.Show();
    }

    private void OnEnable()
    {
        CacheReflection();
        Load();
        RefreshObjectCache();
    }

    private void OnFocus()
    {
        Load();
        RefreshObjectCache();
        Repaint();
    }

    private void OnDisable()
    {
        Save();
    }

    private void OnGUI()
    {

```

```

    DrawHeader();
    DrawToolbar();
    DrawSearch();
    DrawBookmarkList();
}

private void CacheReflection()
{
    hierarchyWindowType = typeof(EditorWindow).Assembly.GetType("UnityEditor.SceneHierarchyWindow");

    if (hierarchyWindowType == null)
        return;

    setExpandedMethod = hierarchyWindowType.GetMethod(
        "SetExpanded",
        BindingFlags.Instance | BindingFlags.Public | BindingFlags.NonPublic
    );
}

private void DrawHeader()
{
    EditorGUILayout.Space(6);

    GUIStyle titleStyle = new GUIStyle(EditorStyles.boldLabel);
    titleStyle.fontSize = 14;

    EditorGUILayout.LabelField("Nova Hierarchy Quick Panel", titleStyle);

    GUIStyle smallStyle = new GUIStyle(EditorStyles.miniLabel);
    smallStyle.wordWrap = true;

    EditorGUILayout.LabelField(
        "Snel naar belangrijke objecten in je Hierarchy.",
        smallStyle
    );

    EditorGUILayout.Space(4);
}

private void DrawToolbar()
{
    using (new EditorGUILayout.HorizontalScope(EditorStyles.toolbar))
    {
        if (GUILayout.Button("Add Selected", EditorStyles.toolbarButton))
        {
            AddSelectedObject();
        }

        if (GUILayout.Button("Refresh", EditorStyles.toolbarButton))
        {
            Load();
            RefreshObjectCache();
            Repaint();
        }

        if (GUILayout.Button("Remove Missing", EditorStyles.toolbarButton))
        {
            RemoveMissingObjects();
        }

        if (GUILayout.Button("Collapse All", EditorStyles.toolbarButton))
        {
            CollapseAllHierarchyObjects();
        }

        GUILayout.FlexibleSpace();

        if (GUILayout.Button("Save", EditorStyles.toolbarButton, GUILayout.Width(45)))

```

```

    {
        Save();
    }

    if (GUILayout.Button("Clear", EditorStyles.toolbarButton, GUILayout.Width(45)))
    {
        ClearAllBookmarks();
    }
}

private void DrawSearch()
{
    EditorGUILayout.Space(6);

    using (new EditorGUILayout.HorizontalScope())
    {
        GUILayout.Label("Search", GUILayout.Width(48));
        searchText = EditorGUILayout.TextField(searchText);

        if (GUILayout.Button("X", GUILayout.Width(24)))
        {
            searchText = "";
            GUI.FocusControl(null);
        }
    }

    EditorGUILayout.Space(4);
}

private void DrawBookmarkList()
{
    if (data.bookmarks.Count == 0)
    {
        EditorGUILayout.HelpBox(
            "Nog geen bookmarks. Selecteer een GameObject en klik op Add Selected.",
            MessageType.Info
        );
        return;
    }

    scrollPosition = EditorGUILayout.BeginScrollView(scrollPosition);

    for (int i = 0; i < data.bookmarks.Count; i++)
    {
        BookmarkData bookmark = data.bookmarks[i];

        if (!PassesSearch(bookmark))
            continue;

        DrawCompactBookmark(i, bookmark);
    }

    EditorGUILayout.EndScrollView();
}

private void DrawCompactBookmark(int index, BookmarkData bookmark)
{
    UnityEngine.Object obj = GetCachedObject(bookmark.globalId);
    bool missing = obj == null;

    Color oldBackground = GUI.backgroundColor;

    if (missing)
    {
        GUI.backgroundColor = new Color(0.75f, 0.35f, 0.35f, 1f);
    }
}

```


VRChat Paper Tablet UI Product Handleiding - v1.6 - StefanieInVR

```
EditorGUILayout.BeginVertical("box");
GUI.backgroundColor = oldBackground;

using (new EditorGUILayout.HorizontalScope())
{
    bookmark.label = EditorGUILayout.TextField(bookmark.label, GUILayout.MinWidth(90));

    GUI.enabled = !missing;

    if (GUILayout.Button("Select", GUILayout.Width(58), GUILayout.Height(22)))
    {
        SelectAndPingObject(obj);
    }

    if (GUILayout.Button("Open", GUILayout.Width(48), GUILayout.Height(22)))
    {
        SelectAndPingObject(obj);
        SetHierarchyExpanded(obj, true);
    }

    if (GUILayout.Button("Close", GUILayout.Width(50), GUILayout.Height(22)))
    {
        SelectAndPingObject(obj);
        SetHierarchyExpanded(obj, false);
    }

    GUI.enabled = true;

    if (GUILayout.Button("X", GUILayout.Width(24), GUILayout.Height(22)))
    {
        data.bookmarks.RemoveAt(index);
        Save();
        RefreshObjectCache();
        GUIUtility.ExitGUI();
    }
}

using (new EditorGUILayout.HorizontalScope())
{
    GUIStyle miniStyle = new GUIStyle(EditorStyles.miniLabel);
    miniStyle.normal.textColor = missing ? new Color(1f, 0.55f, 0.55f) : new Color(0.65f, 0.65f, 0.65f);

    string status = missing ? "Missing object" : obj.name;
    GUILayout.Label(status, miniStyle);

    GUILayout.FlexibleSpace();

    if (GUILayout.Button("Up", EditorStyles.miniButtonLeft, GUILayout.Width(38)))
    {
        MoveBookmark(index, -1);
        GUIUtility.ExitGUI();
    }

    if (GUILayout.Button("Down", EditorStyles.miniButtonMid, GUILayout.Width(48)))
    {
        MoveBookmark(index, 1);
        GUIUtility.ExitGUI();
    }

    GUI.enabled = !missing;

    if (GUILayout.Button("Name", EditorStyles.miniButtonRight, GUILayout.Width(48)))
    {
        bookmark.label = obj.name;
        Save();
    }

    GUI.enabled = true;
}
```

```

    }

    EditorGUILayout.EndVertical();
    EditorGUILayout.Space(2);
}

private bool PassesSearch(BookmarkData bookmark)
{
    if (string.IsNullOrEmpty(searchText))
        return true;

    string label = bookmark.label == null ? "" : bookmark.label;
    return label.ToLowerInvariant().Contains(searchText.ToLowerInvariant());
}

private void AddSelectedObject()
{
    GameObject selected = Selection.activeGameObject;

    if (selected == null)
    {
        EditorUtility.DisplayDialog(
            "Geen GameObject geselecteerd",
            "Selecteer eerst een GameObject in je Hierarchy.",
            "OK"
        );
        return;
    }

    GlobalObjectId globalObjectId = GlobalObjectId.GetGlobalObjectIdSlow(selected);
    string idString = globalObjectId.ToString();

    for (int i = 0; i < data.bookmarks.Count; i++)
    {
        if (data.bookmarks[i].globalId == idString)
        {
            EditorUtility.DisplayDialog(
                "Bestaat al",
                "Dit object staat al in je Nova Hierarchy Quick Panel.",
                "OK"
            );
            return;
        }
    }

    data.bookmarks.Add(new BookmarkData
    {
        label = selected.name,
        globalId = idString
    });

    objectCache[idString] = selected;

    Save();
    Repaint();
}

private void RemoveMissingObjects()
{
    RefreshObjectCache();

    int before = data.bookmarks.Count;

    data.bookmarks.RemoveAll(bookmark => GetCachedObject(bookmark.globalId) == null);

    int removed = before - data.bookmarks.Count;

    Save();

```

```

RefreshObjectCache();

EditorUtility.DisplayDialog(
    "Klaar",
    "Missing bookmarks verwijderd: " + removed,
    "OK"
);
}

private void ClearAllBookmarks()
{
    bool confirm = EditorUtility.DisplayDialog(
        "Alles verwijderen?",
        "Weet je zeker dat je alle bookmarks wilt verwijderen?",
        "Ja",
        "Nee"
    );

    if (!confirm)
        return;

    data.bookmarks.Clear();
    objectCache.Clear();
    Save();
    Repaint();
}

private void MoveBookmark(int index, int direction)
{
    int newIndex = index + direction;

    if (newIndex < 0 || newIndex >= data.bookmarks.Count)
        return;

    BookmarkData item = data.bookmarks[index];
    data.bookmarks.RemoveAt(index);
    data.bookmarks.Insert(newIndex, item);

    Save();
    Repaint();
}

private void RefreshObjectCache()
{
    objectCache.Clear();

    for (int i = 0; i < data.bookmarks.Count; i++)
    {
        BookmarkData bookmark = data.bookmarks[i];

        if (bookmark == null || string.IsNullOrEmpty(bookmark.globalId))
            continue;

        UnityEngine.Object obj = ResolveObjectFromGlobalId(bookmark.globalId);
        objectCache[bookmark.globalId] = obj;
    }
}

private UnityEngine.Object GetCachedObject(string globalId)
{
    if (string.IsNullOrEmpty(globalId))
        return null;

    if (objectCache.TryGetValue(globalId, out UnityEngine.Object obj))
        return obj;

    return null;
}

```

VRChat Paper Tablet UI Product Handleiding - v1.6 - StefanieInVR

```
private UnityEngine.Object ResolveObjectFromGlobalId(string globalIdString)
{
    if (string.IsNullOrEmpty(globalIdString))
        return null;

    if (!GlobalObjectId.TryParse(globalIdString, out GlobalObjectId globalObjectId))
        return null;

    return GlobalObjectId.GlobalObjectIdentifierToObjectSlow(globalObjectId);
}

private void SelectAndPingObject(UnityEngine.Object obj)
{
    if (obj == null)
        return;

    Selection.activeObject = obj;
    EditorGUIUtility.PingObject(obj);
    FocusHierarchyWindow();
}

private void FocusHierarchyWindow()
{
    if (hierarchyWindowType == null)
        return;

    EditorWindow hierarchyWindow = GetWindow(hierarchyWindowType);
    hierarchyWindow.Focus();
}

private void SetHierarchyExpanded(UnityEngine.Object obj, bool expanded)
{
    if (obj == null)
        return;

    if (hierarchyWindowType == null || setExpandedMethod == null)
    {
        Debug.LogWarning("NovaHierarchyQuickPanel: Unity kon de Hierarchy open/close methode niet vinden. Select en Ping werken nog steeds.");
        return;
    }

    EditorWindow hierarchyWindow = GetWindow(hierarchyWindowType);

    try
    {
        setExpandedMethod.Invoke(
            hierarchyWindow,
            new object[]
            {
                obj.GetInstanceID(),
                expanded
            }
        );
    }
    catch (Exception exception)
    {
        Debug.LogWarning("NovaHierarchyQuickPanel: Open/Close werkte niet in deze Unity-versie. Select en Ping werken nog steeds.\n" + exception.Message);
    }
}

private void CollapseAllHierarchyObjects()
{
    if (hierarchyWindowType == null || setExpandedMethod == null)
    {
        Debug.LogWarning("NovaHierarchyQuickPanel: Unity kon de Hierarchy open/close methode niet vinden.");
    }
}
```

```

    return;
}

FocusHierarchyWindow();

int sceneCount = SceneManager.sceneCount;

for (int sceneIndex = 0; sceneIndex < sceneCount; sceneIndex++)
{
    Scene scene = SceneManager.GetSceneAt(sceneIndex);

    if (!scene.isLoaded)
        continue;

    GameObject[] rootObjects = scene.GetRootGameObjects();

    for (int i = 0; i < rootObjects.Length; i++)
    {
        CollapseGameObjectAndChildren(rootObjects[i]);
    }
}

Repaint();
}

private void CollapseGameObjectAndChildren(GameObject gameObject)
{
    if (gameObject == null)
        return;

    SetHierarchyExpanded(gameObject, false);

    Transform transform = gameObject.transform;

    for (int i = 0; i < transform.childCount; i++)
    {
        CollapseGameObjectAndChildren(transform.GetChild(i).gameObject);
    }
}

private void Save()
{
    string json = JsonUtility.ToJson(data);
    EditorPrefs.SetString(EditorPrefsKey, json);
}

private void Load()
{
    string json = EditorPrefs.GetString(EditorPrefsKey, "");

    if (string.IsNullOrEmpty(json))
    {
        data = new BookmarkList();
        return;
    }

    data = JsonUtility.FromJson<BookmarkList>(json);

    if (data == null)
        data = new BookmarkList();

    if (data.bookmarks == null)
        data.bookmarks = new List<BookmarkData>();
}
}
#endif

```

Appendix 22 - NovaHierarchySidebar.cs

```

#if UNITY_EDITOR
using System;
using System.Collections.Generic;
using System.Reflection;
using UnityEditor;
using UnityEngine;
using UnityEngine.SceneManagement;

public class NovaHierarchySidebar : EditorWindow
{
    [Serializable]
    private class BookmarkData
    {
        public string label;
        public string globalId;
    }

    [Serializable]
    private class BookmarkList
    {
        public List<BookmarkData> bookmarks = new List<BookmarkData>();
    }

    private const string EditorPrefsKey = "NovaHierarchyQuickPanel_Bookmarks_v4";

    private BookmarkList data = new BookmarkList();

    private readonly Dictionary<string, UnityEngine.Object> objectCache =
        new Dictionary<string, UnityEngine.Object>();

    private Vector2 scrollPosition;

    private Type hierarchyWindowType;
    private MethodInfo setExpandedMethod;

    [MenuItem("Nova/Editor Tools/Hierarchy Sidebar")]
    public static void OpenWindow()
    {
        NovaHierarchySidebar window = GetWindow<NovaHierarchySidebar>();
        window.titleContent = new GUIContent("Nova Side");
        window.minSize = new Vector2(52, 240);
        window.Show();
    }

    private void OnEnable()
    {
        CacheReflection();
        Load();
        RefreshObjectCache();
    }

    private void OnFocus()
    {
        Load();
        RefreshObjectCache();
        Repaint();
    }

    private void OnGUI()
    {
        DrawTopButtons();
        DrawBookmarkButtons();
    }

    private void CacheReflection()
    {

```

VRChat Paper Tablet UI Product Handleiding - v1.6 - StefanieInVR

```
hierarchyWindowType = typeof(EditorWindow).Assembly.GetType("UnityEditor.SceneHierarchyWindow");

if (hierarchyWindowType == null)
    return;

setExpandedMethod = hierarchyWindowType.GetMethod(
    "SetExpanded",
    BindingFlags.Instance | BindingFlags.Public | BindingFlags.NonPublic
);
}

private void DrawTopButtons()
{
    EditorGUILayout.Space(4);

    using (new EditorGUILayout.HorizontalScope())
    {
        if (GUILayout.Button("+", GUILayout.Width(22), GUILayout.Height(22)))
        {
            AddSelectedObject();
        }

        if (GUILayout.Button("R", GUILayout.Width(22), GUILayout.Height(22)))
        {
            Load();
            RefreshObjectCache();
            Repaint();
        }
    }

    using (new EditorGUILayout.HorizontalScope())
    {
        if (GUILayout.Button("C", GUILayout.Width(22), GUILayout.Height(22)))
        {
            CollapseAllHierarchyObjects();
        }

        if (GUILayout.Button("?", GUILayout.Width(22), GUILayout.Height(22)))
        {
            ShowHelp();
        }
    }

    EditorGUILayout.Space(4);
}

private void DrawBookmarkButtons()
{
    if (data.bookmarks == null || data.bookmarks.Count == 0)
    {
        EditorGUILayout.HelpBox("+", MessageType.None);
        return;
    }

    scrollPosition = EditorGUILayout.BeginScrollView(scrollPosition);

    for (int i = 0; i < data.bookmarks.Count; i++)
    {
        BookmarkData bookmark = data.bookmarks[i];

        if (bookmark == null)
            continue;

        UnityEngine.Object obj = GetCachedObject(bookmark.globalId);
        bool missing = obj == null;

        string buttonText = MakeShortButtonLabel(bookmark.label);
```

VRChat Paper Tablet UI Product Handleiding - v1.6 - StefanieInVR

```
Rect buttonRect = GUILayoutUtility.GetRect(
    40,
    150,
    GUILayout.Width(40),
    GUILayout.Height(150)
);

if (DrawRotatedButton(buttonRect, buttonText, missing))
{
    if (obj != null)
    {
        SelectAndPingObject(obj);
        SetHierarchyExpanded(obj, true);
    }
}

HandleContextClick(buttonRect, i, bookmark, obj);

EditorGUILayout.Space(4);
}

EditorGUILayout.EndScrollView();
}

private string MakeShortButtonLabel(string label)
{
    if (string.IsNullOrEmpty(label))
        return "?";

    string clean = label.Trim();

    if (clean.Length > 20)
        clean = clean.Substring(0, 20);

    return clean;
}

private bool DrawRotatedButton(Rect rect, string text, bool missing)
{
    Event currentEvent = Event.current;
    int controlId = GUIUtility.GetControlID(FocusType.Passive, rect);

    bool clicked = false;

    switch (currentEvent.GetTypeForControl(controlId))
    {
        case EventType.MouseDown:
            if (rect.Contains(currentEvent.mousePosition) && currentEvent.button == 0)
            {
                GUIUtility.hotControl = controlId;
                currentEvent.Use();
            }
            break;

        case EventType.MouseUp:
            if (GUIUtility.hotControl == controlId)
            {
                GUIUtility.hotControl = 0;

                if (rect.Contains(currentEvent.mousePosition))
                {
                    clicked = true;
                    currentEvent.Use();
                }
            }
            break;

        case EventType.Repaint:
```


VRChat Paper Tablet UI Product Handleiding - v1.6 - StefanieInVR

```
        DrawRotatedButtonVisual(rect, text, missing, controlId);
        break;
    }

    return clicked;
}

private void DrawRotatedButtonVisual(Rect rect, string text, bool missing, int controlId)
{
    Event currentEvent = Event.current;

    bool isHot = controlId == GUIUtility.hotControl;
    bool isHover = rect.Contains(currentEvent.mousePosition);

    Color oldBackgroundColor = GUI.backgroundColor;

    if (missing)
    {
        GUI.backgroundColor = new Color(0.70f, 0.25f, 0.25f, 1f);
    }

    GUI.skin.button.Draw(
        rect,
        GUIContent.none,
        isHover,
        isHot,
        false,
        false
    );

    GUI.backgroundColor = oldBackgroundColor;

    Matrix4x4 oldMatrix = GUI.matrix;

    Vector2 pivot = new Vector2(
        rect.x + rect.width * 0.5f,
        rect.y + rect.height * 0.5f
    );

    GUIUtility.RotateAroundPivot(90f, pivot);

    // Important:
    // Because the text is rotated, width and height are swapped.
    // This rect gives the text more safe space inside the button.
    Rect rotatedTextRect = new Rect(
        pivot.x - rect.height * 0.5f + 8f,
        pivot.y - rect.width * 0.5f + 10f,
        rect.height - 16f,
        rect.width - 20f
    );

    GUIStyle textStyle = new GUIStyle(EditorStyles.boldLabel);
    textStyle.alignment = TextAnchor.MiddleCenter;
    textStyle.clipping = TextClipping.Clip;
    textStyle.wordWrap = false;
    textStyle.normal.textColor = EditorGUIUtility.isProSkin
        ? new Color(0.92f, 0.92f, 0.92f, 1f)
        : Color.black;

    // Smaller text than before.
    // This keeps it clearly inside the button.
    textStyle.fontSize = GetBestFittingFontSize(
        textStyle,
        text,
        rotatedTextRect,
        9,
        5
    );
};
```

```

GUI.Label(rotatedTextRect, text, textStyle);

GUI.matrix = oldMatrix;
}

private int GetBestFittingFontSize(
    GUIStyle style,
    string text,
    Rect availableRect,
    int maxFontSize,
    int minFontSize)
{
    if (string.IsNullOrEmpty(text))
        return maxFontSize;

    GUIContent content = new GUIContent(text);

    for (int size = maxFontSize; size >= minFontSize; size--)
    {
        style.fontSize = size;

        Vector2 textSize = style.CalcSize(content);

        if (textSize.x <= availableRect.width - 2f &&
            textSize.y <= availableRect.height - 2f)
        {
            return size;
        }
    }

    return minFontSize;
}

private void HandleContextClick(Rect rect, int index, BookmarkData bookmark, UnityEngine.Object obj)
{
    Event currentEvent = Event.current;

    if (currentEvent.type != EventType.ContextClick)
        return;

    if (!rect.Contains(currentEvent.mousePosition))
        return;

    GenericMenu menu = new GenericMenu();

    if (obj != null)
    {
        menu.AddItem(new GUIContent("Select"), false, () =>
        {
            SelectAndPingObject(obj);
        });

        menu.AddItem(new GUIContent("Open"), false, () =>
        {
            SelectAndPingObject(obj);
            SetHierarchyExpanded(obj, true);
        });

        menu.AddItem(new GUIContent("Close"), false, () =>
        {
            SelectAndPingObject(obj);
            SetHierarchyExpanded(obj, false);
        });

        menu.AddItem(new GUIContent("Use Object Name"), false, () =>
        {
            bookmark.label = obj.name;

```

```

        Save();
        Repaint();
    });
}
else
{
    menu.AddDisabledItem(new GUIContent("Missing object"));
}

menu.AddSeparator("");

menu.AddItem(new GUIContent("Move Up"), false, () =>
{
    MoveBookmark(index, -1);
});

menu.AddItem(new GUIContent("Move Down"), false, () =>
{
    MoveBookmark(index, 1);
});

menu.AddSeparator("");

menu.AddItem(new GUIContent("Remove Bookmark"), false, () =>
{
    data.bookmarks.RemoveAt(index);
    Save();
    RefreshObjectCache();
    Repaint();
});

menu.ShowAsContext();
currentEvent.Use();
}

private void AddSelectedObject()
{
    Load();

    GameObject selected = Selection.activeGameObject;

    if (selected == null)
    {
        EditorUtility.DisplayDialog(
            "Geen GameObject geselecteerd",
            "Selecteer eerst een GameObject in je Hierarchy.",
            "OK"
        );
        return;
    }

    GlobalObjectId globalObjectId = GlobalObjectId.GetGlobalObjectIdSlow(selected);
    string idString = globalObjectId.ToString();

    for (int i = 0; i < data.bookmarks.Count; i++)
    {
        if (data.bookmarks[i].globalId == idString)
        {
            EditorUtility.DisplayDialog(
                "Bestaat al",
                "Dit object staat al in je Nova Hierarchy bookmarks.",
                "OK"
            );
            return;
        }
    }
}

data.bookmarks.Add(new BookmarkData

```

```

{
    label = selected.name,
    globalId = idString
});

Save();
RefreshObjectCache();
Repaint();
}

private void MoveBookmark(int index, int direction)
{
    int newIndex = index + direction;

    if (newIndex < 0 || newIndex >= data.bookmarks.Count)
        return;

    BookmarkData item = data.bookmarks[index];
    data.bookmarks.RemoveAt(index);
    data.bookmarks.Insert(newIndex, item);

    Save();
    RefreshObjectCache();
    Repaint();
}

private void RefreshObjectCache()
{
    objectCache.Clear();

    if (data.bookmarks == null)
        return;

    for (int i = 0; i < data.bookmarks.Count; i++)
    {
        BookmarkData bookmark = data.bookmarks[i];

        if (bookmark == null || string.IsNullOrEmpty(bookmark.globalId))
            continue;

        UnityEngine.Object obj = ResolveObjectFromGlobalId(bookmark.globalId);
        objectCache[bookmark.globalId] = obj;
    }
}

private UnityEngine.Object GetCachedObject(string globalId)
{
    if (string.IsNullOrEmpty(globalId))
        return null;

    if (objectCache.TryGetValue(globalId, out UnityEngine.Object obj))
        return obj;

    return null;
}

private UnityEngine.Object ResolveObjectFromGlobalId(string globalIdString)
{
    if (string.IsNullOrEmpty(globalIdString))
        return null;

    if (!GlobalObjectId.TryParse(globalIdString, out GlobalObjectId globalObjectId))
        return null;

    return GlobalObjectId.GlobalObjectIdentifierToObjectSlow(globalObjectId);
}

private void SelectAndPingObject(UnityEngine.Object obj)

```

VRChat Paper Tablet UI Product Handleiding - v1.6 - StefanieInVR

```
{
    if (obj == null)
        return;

    Selection.activeObject = obj;
    EditorGUIUtility.PingObject(obj);
    FocusHierarchyWindow();
}

private void FocusHierarchyWindow()
{
    if (hierarchyWindowType == null)
        return;

    EditorWindow hierarchyWindow = GetWindow(hierarchyWindowType);
    hierarchyWindow.Focus();
}

private void SetHierarchyExpanded(UnityEngine.Object obj, bool expanded)
{
    if (obj == null)
        return;

    if (hierarchyWindowType == null || setExpandedMethod == null)
    {
        Debug.LogWarning("NovaHierarchySidebar: Unity kon de Hierarchy open/close methode niet vinden. Select en Ping werken nog steeds.");
        return;
    }

    EditorWindow hierarchyWindow = GetWindow(hierarchyWindowType);

    try
    {
        setExpandedMethod.Invoke(
            hierarchyWindow,
            new object[]
            {
                obj.GetInstanceID(),
                expanded
            }
        );
    }
    catch (Exception exception)
    {
        Debug.LogWarning("NovaHierarchySidebar: Open/Close werkte niet in deze Unity-versie. Select en Ping werken nog steeds.\n" + exception.Message);
    }
}

private void CollapseAllHierarchyObjects()
{
    if (hierarchyWindowType == null || setExpandedMethod == null)
    {
        Debug.LogWarning("NovaHierarchySidebar: Unity kon de Hierarchy open/close methode niet vinden.");
        return;
    }

    FocusHierarchyWindow();

    int sceneCount = SceneManager.sceneCount;

    for (int sceneIndex = 0; sceneIndex < sceneCount; sceneIndex++)
    {
        Scene scene = SceneManager.GetSceneAt(sceneIndex);

        if (!scene.isLoaded)
            continue;
    }
}
```

VRChat Paper Tablet UI Product Handleiding - v1.6 - StefanieInVR

```
GameObject[] rootObjects = scene.GetRootGameObjects();

for (int i = 0; i < rootObjects.Length; i++)
{
    CollapseGameObjectAndChildren(rootObjects[i]);
}

Repaint();
}

private void CollapseGameObjectAndChildren(GameObject gameObject)
{
    if (gameObject == null)
        return;

    SetHierarchyExpanded(gameObject, false);

    Transform transform = gameObject.transform;

    for (int i = 0; i < transform.childCount; i++)
    {
        CollapseGameObjectAndChildren(transform.GetChild(i).gameObject);
    }
}

private void ShowHelp()
{
    EditorUtility.DisplayDialog(
        "Nova Hierarchy Sidebar",
        "Knoppen:\n\n+ = selected object toevoegen\nR = refresh\nC = alles dichtklappen\n? = uitleg\n\nBookmark knop:\nLeft click = selecteer en open\nRight click = extra opties zoals close, remove, up/down",
        "OK"
    );
}

private void Save()
{
    string json = JsonUtility.ToJson(data);
    EditorPrefs.SetString(EditorPrefsKey, json);
}

private void Load()
{
    string json = EditorPrefs.GetString(EditorPrefsKey, "");

    if (string.IsNullOrEmpty(json))
    {
        data = new BookmarkList();
        return;
    }

    data = JsonUtility.FromJson<BookmarkList>(json);

    if (data == null)
        data = new BookmarkList();

    if (data.bookmarks == null)
        data.bookmarks = new List<BookmarkData>();
}
}
#endif
```

Appendix 23 - NovaSceneTabletAnalyzer.cs

```

#if UNITY_EDITOR
using System;
using System.Collections.Generic;
using System.IO;
using System.Text;
using UnityEditor;
using UnityEditor.SceneManagement;
using UnityEngine;
using UnityEngine.SceneManagement;

public class NovaSceneTabletAnalyzer : EditorWindow
{
    private bool includeInactiveObjects = true;
    private bool includeFullScriptCode = true;
    private bool includeSerializedFields = true;
    private bool includeMaterials = true;
    private bool includeUIInfo = true;

    private Vector2 scroll;

    [MenuItem("Nova/Scene Tools/Analyze Current Scene")]
    public static void ShowWindow()
    {
        NovaSceneTabletAnalyzer window = GetWindow<NovaSceneTabletAnalyzer>();
        window.titleContent = new GUIContent("Nova Scene Analyzer");
        window.minSize = new Vector2(420, 320);
        window.Show();
    }

    private void OnGUI()
    {
        EditorGUILayout.LabelField("Nova Scene Analyzer", EditorStyles.boldLabel);
        EditorGUILayout.HelpBox(
            "Deze tool leest de huidige scene uit en maakt TXT, Markdown, JSON en CSV rapporten. " +
            "Handig om aan Nova te geven zodat zij precies kan zien wat gekoppeld is.",
            MessageType.Info
        );

        scroll = EditorGUILayout.BeginScrollView(scroll);

        includeInactiveObjects = EditorGUILayout.ToggleLeft("Include inactive GameObjects", includeInactiveObjects);
        includeSerializedFields = EditorGUILayout.ToggleLeft("Include serialized fields and references", includeSerializedFields);
        includeMaterials = EditorGUILayout.ToggleLeft("Include renderer/material info", includeMaterials);
        includeUIInfo = EditorGUILayout.ToggleLeft("Include UI Button/Slider/Toggle info", includeUIInfo);
        includeFullScriptCode = EditorGUILayout.ToggleLeft("Include full script code at end", includeFullScriptCode);

        EditorGUILayout.Space(12);

        if (GUILayout.Button("Analyze Current Scene", GUILayout.Height(38)))
        {
            AnalyzeCurrentScene();
        }

        EditorGUILayout.Space(12);

        EditorGUILayout.HelpBox(
            "Output komt in:\nAssets/NovaSceneReports/\n\nStuur vooral het .txt of .md bestand naar Nova.",
            MessageType.None
        );

        EditorGUILayout.EndScrollView();
    }

    private void AnalyzeCurrentScene()
    {
        Scene scene = SceneManager.GetActiveScene();

```

```

if (!scene.IsValid())
{
    EditorUtility.DisplayDialog("Nova Scene Analyzer", "No valid active scene found.", "OK");
    return;
}

string folder = "Assets/NovaSceneReports";
if (!Directory.Exists(folder))
{
    Directory.CreateDirectory(folder);
}

string safeSceneName = MakeSafeFileName(scene.name);
string timestamp = DateTime.Now.ToString("yyyy-MM-dd_HH-mm-ss");

string basePath = Path.Combine(folder, safeSceneName + "_SceneReport_" + timestamp);

List<GameObject> allObjects = GetAllSceneObjects(scene, includeInactiveObjects);
List<ScriptInfo> usedScripts = CollectUsedScripts(allObjects);

string txt = BuildTextReport(scene, allObjects, usedScripts);
string md = BuildMarkdownReport(scene, allObjects, usedScripts);
string json = BuildJsonReport(scene, allObjects, usedScripts);
string csv = BuildCsvReport(allObjects);

File.WriteAllText(basePath + ".txt", txt, Encoding.UTF8);
File.WriteAllText(basePath + ".md", md, Encoding.UTF8);
File.WriteAllText(basePath + ".json", json, Encoding.UTF8);
File.WriteAllText(basePath + ".csv", csv, Encoding.UTF8);

AssetDatabase.Refresh();

EditorUtility.DisplayDialog(
    "Nova Scene Analyzer",
    "Scene analysis complete.\n\nCreated files:\n" +
    basePath + ".txt\n" +
    basePath + ".md\n" +
    basePath + ".json\n" +
    basePath + ".csv",
    "OK"
);

Debug.Log("[Nova Scene Analyzer] Report created at: " + basePath);
}

private List<GameObject> GetAllSceneObjects(Scene scene, bool includeInactive)
{
    List<GameObject> result = new List<GameObject>();

    GameObject[] roots = scene.GetRootGameObjects();

    foreach (GameObject root in roots)
    {
        AddObjectRecursive(root, result, includeInactive);
    }

    result.Sort((a, b) => string.Compare(GetHierarchyPath(a), GetHierarchyPath(b), StringComparison.OrdinalIgnoreCase));

    return result;
}

private void AddObjectRecursive(GameObject obj, List<GameObject> list, bool includeInactive)
{
    if (obj == null) return;

    if (includeInactive || obj.activeInHierarchy || obj.activeSelf)
    {

```



```

        list.Add(obj);
    }

    for (int i = 0; i < obj.transform.childCount; i++)
    {
        AddObjectRecursive(obj.transform.GetChild(i).gameObject, list, includeInactive);
    }
}

private List<ScriptInfo> CollectUsedScripts(List<GameObject> objects)
{
    Dictionary<string, ScriptInfo> scripts = new Dictionary<string, ScriptInfo>();

    foreach (GameObject obj in objects)
    {
        Component[] components = obj.GetComponents<Component>();

        foreach (Component component in components)
        {
            if (component == null) continue;

            MonoBehaviour mono = component as MonoBehaviour;
            if (mono == null) continue;

            MonoScript monoScript = MonoScript.FromMonoBehaviour(mono);
            if (monoScript == null) continue;

            string assetPath = AssetDatabase.GetAssetPath(monoScript);
            if (string.IsNullOrEmpty(assetPath)) continue;

            if (!scripts.ContainsKey(assetPath))
            {
                ScriptInfo info = new ScriptInfo();
                info.className = monoScript.GetClass() != null ? monoScript.GetClass().Name : monoScript.name;
                info.assetPath = assetPath;
                info.usedOnObjects = new List<string>();

                if (includeFullScriptCode)
                {
                    try
                    {
                        info.sourceCode = File.ReadAllText(assetPath);
                    }
                    catch
                    {
                        info.sourceCode = "[Could not read script file]";
                    }
                }

                scripts.Add(assetPath, info);
            }

            scripts[assetPath].usedOnObjects.Add(GetHierarchyPath(obj));
        }
    }

    List<ScriptInfo> result = new List<ScriptInfo>(scripts.Values);
    result.Sort((a, b) => string.Compare(a.className, b.className, StringComparison.OrdinalIgnoreCase));
    return result;
}

private string BuildTextReport(Scene scene, List<GameObject> objects, List<ScriptInfo> usedScripts)
{
    StringBuilder sb = new StringBuilder();

    sb.AppendLine("NOVA SCENE ANALYSIS REPORT");
    sb.AppendLine("=====");
    sb.AppendLine();

```

VRChat Paper Tablet UI Product Handleiding - v1.6 - StefanieInVR

```
sb.AppendLine("Scene: " + scene.name);
sb.AppendLine("Scene path: " + scene.path);
sb.AppendLine("Generated: " + DateTime.Now.ToString("yyyy-MM-dd HH:mm:ss"));
sb.AppendLine("Unity version: " + Application.unityVersion);
sb.AppendLine("Object count: " + objects.Count);
sb.AppendLine("Script count: " + usedScripts.Count);
sb.AppendLine();

sb.AppendLine("IMPORTANT");
sb.AppendLine("-----");
sb.AppendLine("This report shows GameObjects, components, serialized fields, object references and used script code.");
sb.AppendLine("Use this file to let Nova inspect the current scene setup.");
sb.AppendLine();

sb.AppendLine("ROOT OBJECTS");
sb.AppendLine("-----");

foreach (GameObject root in scene.GetRootGameObjects())
{
    sb.AppendLine("- " + root.name);
}

sb.AppendLine();
sb.AppendLine("HIERARCHY AND COMPONENTS");
sb.AppendLine("=====");
sb.AppendLine();

foreach (GameObject obj in objects)
{
    AppendGameObjectText(sb, obj);
}

sb.AppendLine();
sb.AppendLine("USED SCRIPTS");
sb.AppendLine("=====");
sb.AppendLine();

foreach (ScriptInfo script in usedScripts)
{
    sb.AppendLine("Script: " + script.className);
    sb.AppendLine("Path: " + script.assetPath);
    sb.AppendLine("Used on:");

    foreach (string objPath in script.usedOnObjects)
    {
        sb.AppendLine(" - " + objPath);
    }

    sb.AppendLine();
}

if (includeFullScriptCode)
{
    sb.AppendLine();
    sb.AppendLine("FULL SCRIPT CODE");
    sb.AppendLine("=====");
    sb.AppendLine();

    foreach (ScriptInfo script in usedScripts)
    {
        sb.AppendLine("=====");
        sb.AppendLine("SCRIPT: " + script.className);
        sb.AppendLine("PATH: " + script.assetPath);
        sb.AppendLine("=====");
        sb.AppendLine();
        sb.AppendLine(script.sourceCode);
        sb.AppendLine();
    }
}
```

```

    }

    return sb.ToString();
}

private string BuildMarkdownReport(Scene scene, List<GameObject> objects, List<ScriptInfo> usedScripts)
{
    StringBuilder sb = new StringBuilder();

    sb.AppendLine("# Nova Scene Analysis Report");
    sb.AppendLine();
    sb.AppendLine("## Scene Info");
    sb.AppendLine();
    sb.AppendLine("- Scene: " + scene.name + "");
    sb.AppendLine("- Scene path: " + scene.path + "");
    sb.AppendLine("- Generated: " + DateTime.Now.ToString("yyyy-MM-dd HH:mm:ss") + "");
    sb.AppendLine("- Unity version: " + Application.unityVersion + "");
    sb.AppendLine("- Object count: " + objects.Count + "");
    sb.AppendLine("- Script count: " + usedScripts.Count + "");
    sb.AppendLine();

    sb.AppendLine("## Hierarchy And Components");
    sb.AppendLine();

    foreach (GameObject obj in objects)
    {
        sb.AppendLine("### " + EscapeMarkdown(GetHierarchyPath(obj)));
        sb.AppendLine();
        sb.AppendLine("- Active Self: " + obj.activeSelf + "");
        sb.AppendLine("- Active In Hierarchy: " + obj.activeInHierarchy + "");
        sb.AppendLine("- Tag: " + obj.tag + "");
        sb.AppendLine("- Layer: " + LayerMask.LayerToName(obj.layer) + "");
        sb.AppendLine("- Transform:");
        sb.AppendLine("  - Local Position: " + FormatVector3(obj.transform.localPosition) + "");
        sb.AppendLine("  - Local Rotation: " + FormatVector3(obj.transform.localEulerAngles) + "");
        sb.AppendLine("  - Local Scale: " + FormatVector3(obj.transform.localScale) + "");
        sb.AppendLine();

        Component[] components = obj.GetComponents<Component>();
        sb.AppendLine("Components:");

        foreach (Component component in components)
        {
            if (component == null)
            {
                sb.AppendLine("- Missing Script / Missing Component");
                continue;
            }

            sb.AppendLine("- " + component.GetType().Name + "");

            if (includeSerializedFields)
            {
                {
                    List<string> fields = GetSerializedFields(component);
                    foreach (string field in fields)
                    {
                        sb.AppendLine("  - " + field);
                    }
                }
            }
        }

        sb.AppendLine();
    }

    sb.AppendLine("## Used Scripts");
    sb.AppendLine();

    foreach (ScriptInfo script in usedScripts)

```

```

{
    sb.AppendLine("### " + script.className);
    sb.AppendLine();
    sb.AppendLine("- Path: " + script.assetPath + "");
    sb.AppendLine("- Used on:");

    foreach (string objPath in script.usedOnObjects)
    {
        sb.AppendLine(" - " + objPath + "");
    }

    sb.AppendLine();
}

if (includeFullScriptCode)
{
    sb.AppendLine("## Full Script Code");
    sb.AppendLine();

    foreach (ScriptInfo script in usedScripts)
    {
        sb.AppendLine("### " + script.className);
        sb.AppendLine();
        sb.AppendLine("Path: " + script.assetPath + "");
        sb.AppendLine();
        sb.AppendLine("`csharp");
        sb.AppendLine(script.sourceCode);
        sb.AppendLine("```");
        sb.AppendLine();
    }
}

return sb.ToString();
}

private string BuildJsonReport(Scene scene, List<GameObject> objects, List<ScriptInfo> usedScripts)
{
    SceneReportJson report = new SceneReportJson();
    report.sceneName = scene.name;
    report.scenePath = scene.path;
    report.generated = DateTime.Now.ToString("yyyy-MM-dd HH:mm:ss");
    report.unityVersion = Application.unityVersion;
    report.objects = new List<GameObjectJson>();
    report.scripts = usedScripts;

    foreach (GameObject obj in objects)
    {
        GameObjectJson objJson = new GameObjectJson();
        objJson.name = obj.name;
        objJson.path = GetHierarchyPath(obj);
        objJson.activeSelf = obj.activeSelf;
        objJson.activeInHierarchy = obj.activeInHierarchy;
        objJson.tag = obj.tag;
        objJson.layer = LayerMask.LayerToName(obj.layer);
        objJson.localPosition = FormatVector3(obj.transform.localPosition);
        objJson.localRotation = FormatVector3(obj.transform.localEulerAngles);
        objJson.localScale = FormatVector3(obj.transform.localScale);
        objJson.components = new List<ComponentJson>();

        Component[] components = obj.GetComponents<Component>();
        foreach (Component component in components)
        {
            ComponentJson compJson = new ComponentJson();

            if (component == null)
            {
                compJson.type = "Missing Script / Missing Component";
                compJson.fields = new List<string>();
            }
        }
    }
}

```

```

    }
    else
    {
        compJson.type = component.GetType().Name;
        compJson.fields = includeSerializedFields ? GetSerializedFields(component) : new List<string>();
    }

    objJson.components.Add(compJson);
}

report.objects.Add(objJson);
}

return JsonUtility.ToJson(report, true);
}

private string BuildCsvReport(List<GameObject> objects)
{
    StringBuilder sb = new StringBuilder();

    sb.AppendLine("Path,Name,ActiveSelf,ActiveInHierarchy,Tag,Layer,ComponentCount,Components");

    foreach (GameObject obj in objects)
    {
        Component[] components = obj.GetComponents<Component>();
        List<string> componentNames = new List<string>();

        foreach (Component component in components)
        {
            if (component == null)
            {
                componentNames.Add("Missing Script");
            }
            else
            {
                componentNames.Add(component.GetType().Name);
            }
        }

        sb.Append(EscapeCsv(GetHierarchyPath(obj)));
        sb.Append(",");
        sb.Append(EscapeCsv(obj.name));
        sb.Append(",");
        sb.Append(EscapeCsv(obj.activeSelf.ToString()));
        sb.Append(",");
        sb.Append(EscapeCsv(obj.activeInHierarchy.ToString()));
        sb.Append(",");
        sb.Append(EscapeCsv(obj.tag));
        sb.Append(",");
        sb.Append(EscapeCsv(LayerMask.LayerToName(obj.layer)));
        sb.Append(",");
        sb.Append(EscapeCsv(components.Length.ToString()));
        sb.Append(",");
        sb.Append(EscapeCsv(string.Join(" | ", componentNames.ToArray())));
        sb.AppendLine();
    }

    return sb.ToString();
}

private void AppendGameObjectText(StringBuilder sb, GameObject obj)
{
    sb.AppendLine("-----");
    sb.AppendLine("GameObject: " + GetHierarchyPath(obj));
    sb.AppendLine("-----");
    sb.AppendLine("Name: " + obj.name);
    sb.AppendLine("Active Self: " + obj.activeSelf);
    sb.AppendLine("Active In Hierarchy: " + obj.activeInHierarchy);

```

VRChat Paper Tablet UI Product Handleiding - v1.6 - StefanieInVR

```
sb.AppendLine("Tag: " + obj.tag);
sb.AppendLine("Layer: " + LayerMask.LayerToName(obj.layer));
sb.AppendLine("Local Position: " + FormatVector3(obj.transform.localPosition));
sb.AppendLine("Local Rotation: " + FormatVector3(obj.transform.localEulerAngles));
sb.AppendLine("Local Scale: " + FormatVector3(obj.transform.localScale));
sb.AppendLine();

Component[] components = obj.GetComponents<Component>();

sb.AppendLine("Components:");

foreach (Component component in components)
{
    if (component == null)
    {
        sb.AppendLine(" - Missing Script / Missing Component");
        continue;
    }

    sb.AppendLine(" - " + component.GetType().FullName);

    if (includeSerializedFields)
    {
        List<string> fields = GetSerializedFields(component);
        foreach (string field in fields)
        {
            sb.AppendLine("    " + field);
        }
    }
}

sb.AppendLine();
}

private List<string> GetSerializedFields(Component component)
{
    List<string> result = new List<string>();

    if (component == null) return result;

    try
    {
        SerializedObject serializedObject = new SerializedObject(component);
        SerializedProperty property = serializedObject.GetIterator();

        bool enterChildren = true;

        while (property.NextVisible(enterChildren))
        {
            enterChildren = false;

            if (property.name == "m_Script")
            {
                UnityEngine.Object scriptObj = property.objectReferenceValue;
                string scriptPath = scriptObj != null ? AssetDatabase.GetAssetPath(scriptObj) : "None";
                result.Add("Script = " + scriptPath);
                continue;
            }

            string line = property.displayName + " (" + property.name + ") = " + SerializedPropertyToString(property);

            result.Add(line);
        }
    }
    catch (Exception ex)
    {
        result.Add("[Could not read serialized fields: " + ex.Message + "];");
    }
}
```

```

    return result;
}

private string SerializedPropertyToString(SerializedProperty property)
{
    switch (property.propertyType)
    {
        case SerializedPropertyType.Integer:
            return property.intValue.ToString();

        case SerializedPropertyType.Boolean:
            return property.boolValue.ToString();

        case SerializedPropertyType.Float:
            return property.floatValue.ToString();

        case SerializedPropertyType.String:
            return "\"" + property.stringValue + "\"";

        case SerializedPropertyType.Color:
            return property.colorValue.ToString();

        case SerializedPropertyType.ObjectReference:
            return ObjectReferenceToString(property.objectReferenceValue);

        case SerializedPropertyType.LayerMask:
            return property.intValue.ToString();

        case SerializedPropertyType.Enum:
            return property.enumDisplayNames != null && property.enumValueIndex >= 0 && property.enumValueIndex <
property.enumDisplayNames.Length
                ? property.enumDisplayNames[property.enumValueIndex]
                : property.enumValueIndex.ToString();

        case SerializedPropertyType.Vector2:
            return property.vector2Value.ToString();

        case SerializedPropertyType.Vector3:
            return property.vector3Value.ToString();

        case SerializedPropertyType.Vector4:
            return property.vector4Value.ToString();

        case SerializedPropertyType.Rect:
            return property.rectValue.ToString();

        case SerializedPropertyType.ArraySize:
            return property.intValue.ToString();

        case SerializedPropertyType.Character:
            return property.intValue.ToString();

        case SerializedPropertyType.AnimationCurve:
            return property.animationCurveValue != null ? property.animationCurveValue.ToString() : "None";

        case SerializedPropertyType.Bounds:
            return property.boundsValue.ToString();

        case SerializedPropertyType.Quaternion:
            return property.quaternionValue.eulerAngles.ToString();

        case SerializedPropertyType.ExposedReference:
            return ObjectReferenceToString(property.exposedReferenceValue);

        case SerializedPropertyType.FixedBufferSize:
            return property.intValue.ToString();
    }
}

```

VRChat Paper Tablet UI Product Handleiding - v1.6 - StefanieInVR

```
case SerializedPropertyType.Vector2Int:
    return property.vector2IntValue.ToString();

case SerializedPropertyType.Vector3Int:
    return property.vector3IntValue.ToString();

case SerializedPropertyType.RectInt:
    return property.rectIntValue.ToString();

case SerializedPropertyType.BoundsInt:
    return property.boundsIntValue.ToString();

case SerializedPropertyType.ManagedReference:
    return property.managedReferenceFullTypeName;

case SerializedPropertyType.Generic:
    if (property.isArray)
    {
        return "Array, size = " + property.arraySize;
    }
    return "Generic";

default:
    return "[Unsupported property type: " + property.propertyType + "]";
}
}

private string ObjectReferenceToString(UnityEngine.Object obj)
{
    if (obj == null) return "None";

    GameObject go = obj as GameObject;
    if (go != null)
    {
        return "GameObject: " + GetHierarchyPath(go);
    }

    Component component = obj as Component;
    if (component != null)
    {
        return "Component: " + component.GetType().Name + " on " + GetHierarchyPath(component.gameObject);
    }

    string assetPath = AssetDatabase.GetAssetPath(obj);
    if (!string.IsNullOrEmpty(assetPath))
    {
        return obj.GetType().Name + ": " + obj.name + " | AssetPath: " + assetPath;
    }

    return obj.GetType().Name + ": " + obj.name;
}

private string GetHierarchyPath(GameObject obj)
{
    if (obj == null) return "None";

    string path = obj.name;
    Transform current = obj.transform.parent;

    while (current != null)
    {
        path = current.name + "/" + path;
        current = current.parent;
    }

    return path;
}
```


VRChat Paper Tablet UI Product Handleiding - v1.6 - StefanieInVR

```
private string FormatVector3(Vector3 value)
{
    return value.x.ToString("0.###") + ", " + value.y.ToString("0.###") + ", " + value.z.ToString("0.###");
}
```

```
private string MakeSafeFileName(string input)
{
    if (string.IsNullOrEmpty(input)) return "UntitledScene";

    foreach (char c in Path.GetInvalidFileNameChars())
    {
        input = input.Replace(c, '_');
    }

    return input;
}
```

```
private string EscapeCsv(string value)
{
    if (value == null) return "";

    value = value.Replace("\"", "\\\"");

    if (value.Contains(",") || value.Contains("\"") || value.Contains("\n") || value.Contains("\r"))
    {
        value = "\"" + value + "\"";
    }

    return value;
}
```

```
private string EscapeMarkdown(string value)
{
    if (string.IsNullOrEmpty(value)) return "";
    return value.Replace("#", "\\#");
}
```

```
[Serializable]
public class SceneReportJson
{
    public string sceneName;
    public string scenePath;
    public string generated;
    public string unityVersion;
    public List<GameObjectJson> objects;
    public List<ScriptInfo> scripts;
}
```

```
[Serializable]
public class GameObjectJson
{
    public string name;
    public string path;
    public bool activeSelf;
    public bool activeInHierarchy;
    public string tag;
    public string layer;
    public string localPosition;
    public string localRotation;
    public string localScale;
    public List<ComponentJson> components;
}
```

```
[Serializable]
public class ComponentJson
{
    public string type;
    public List<string> fields;
}
```

```
}

[Serializable]
public class ScriptInfo
{
    public string className;
    public string assetPath;
    public List<string> usedOnObjects;
    public string sourceCode;
}
#endif
```